

Agents with Small Language Models

Optimization Techniques for Reliable Reasoning and Tool Calling

Master Thesis

Paulo Ricardo Beckhauser de Araujo



Agents with Small Language Models

Optimization Techniques for Reliable Reasoning and Tool Calling

Master Thesis

July, 2026

By

Paulo Ricardo Beckhauser de Araujo

Copyright: Reproduction of this publication in whole or in part must include the customary bibliographic citation, including author attribution, report title, etc.

Cover photo: Vibeke Hempler, 2012

Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk

ISSN: [0000-0000] (electronic version)

ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)

ISBN: [000-00-0000-000-0] (printed version)

Approval

This Master's thesis was prepared for the fulfillment for the degree Master of Science in Engineering, MSc Autonomous Systems at the Technical University of Denmark (DTU). The thesis was prepared by Paulo Ricardo Beckhauser de Araujo (student ID: s242779) under supervision of Nicki Skafte Detlefsen, Professor at DTU Compute. The project period lasted 6 months from January 5th, 2026 until July 5th, 2026, equal to a workload of 35 ECTS points.

The reader is expected to be familiar with the basic concepts of linear algebra, programming languages, deep learning and generative AI.

Paulo Ricardo Beckhauser de Araujo - s242779

.....
Signature

.....
Date

Abstract

Frontier language models have become the default backbone for agentic systems, yet their inference costs and dependence on external APIs make them impractical for latency-sensitive or resource-constrained deployments. Small language models (1B–8B parameters) can run on consumer hardware, but without optimization they fail on the structured output and tool-calling tasks that agentic pipelines require. This thesis investigates which optimization techniques are sufficient to make a small language model reliably call tools, and at what accuracy cost each technique comes.

A cumulative ablation study is conducted on `Qwen/Qwen2.5-7B-Instruct` across five techniques, evaluated on the Berkeley Function Calling Leaderboard v4 `simple_python` category (400 test cases, AST accuracy). The unoptimized model achieves 1.5%, approximately 98 percentage points below frontier models. Constrained decoding alone raises accuracy to 72.75%, matching GPT-4.1 (72.67%) and Claude Sonnet 4.5 (72.58%), at zero additional inference cost. AWQ INT4 quantization reduces model memory from 14.25 GiB to 5.20 GiB with a loss of only 0.5 percentage points (72.25%). Few-shot prompting and chain-of-thought reasoning both reduce accuracy, by 2.5 and 6.5 percentage points respectively. Retrieval-augmented generation with top-5 retrieval reduces accuracy by 24 percentage points despite 97.2% recall, due to function disambiguation failure. LoRA fine-tuning on a general-purpose function-calling dataset produces a 3 percentage point regression under constrained decoding, attributed to a mismatch between the training output format and the evaluation pipeline.

The primary finding is that the unoptimized performance gap is a format compliance gap rather than a reasoning gap: 394 of 400 failures in the unoptimized configuration are malformed outputs, not wrong answers. Constrained decoding resolves this gap structurally, and the quantized configuration (72.25%, 5.20 GiB) is production-viable on a consumer NVIDIA RTX 4090. Prompting interventions cannot close the residual 27% failure rate because those errors are in the model’s learned value conventions, not in output format. Fine-tuning with exact format alignment between training and inference remains the direct path to accuracy above the no-training ceiling.

Acknowledgements

This thesis was completed with the guidance and support of **Nicki Skafte Detlefsen**, Associate Professor at the Technical University of Denmark, who supervised this work.

I would also like to thank the professors and friends who were part of my journey throughout my master's degree, and above all, my family for their unwavering support along the way.

Contents

Preface	ii
Abstract	iii
Acknowledgements	iv
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Contributions	3
1.5 Thesis Structure	3
2 Background	5
2.1 Language Models	5
2.2 Agentic Systems	6
2.3 Constrained Decoding	7
2.4 Prompt Engineering and Inference-Time Compute	8
2.5 Retrieval-Augmented Generation	8
2.6 Parameter-Efficient Fine-Tuning	9
2.7 Model Quantization	9
2.8 Related Work	10
3 Methodology	14
3.1 Experimental Design	14
3.2 Evaluation Framework	15
3.3 Models	15
3.4 Baseline Definition	16
3.5 Constrained Decoding	16
3.6 Quantization	16
3.7 Prompt Engineering and Few-Shot	17
3.8 Retrieval-Augmented Generation	17
3.9 LoRA Fine-Tuning	18
4 Results	19
4.1 Baseline Performance	19
4.2 No-Training Methods	19
4.3 Training-Based Methods	23
4.4 Combined Configurations	23
4.5 Comparison with Frontier Models	26
4.6 Agentic Evaluation	27
4.7 Model-Size Scaling	28
4.8 Extended Function-Calling Categories	28
4.9 Quantized Fine-Tuning	29
5 Discussion	30
5.1 Impact of Constrained Decoding vs. Model Scale	30
5.2 Cost-Benefit Analysis of Each Method	30
5.3 Limitations	32

5.4	Practical Implications	32
6	Conclusion	35
6.1	Answering the Research Questions	35
6.2	Future Work	35
	Bibliography	37
A	Generative AI Disclosure	39
B	Hyperparameters and Training Details	40
C	Full Benchmark Results	41

1 Introduction

1.1 Motivation

Large language models (LLMs) have become the dominant backbone for agentic systems, powering applications that reason over user requests, decompose tasks, and call external tools to produce structured outputs. Frontier models such as GPT-4 and Claude demonstrate strong performance on these tasks, but their deployment comes with significant trade-offs: high inference costs, dependency on external APIs, increased latency, and limited control over data privacy. For many real-world applications, these constraints make frontier models impractical.

Small language models (SLMs), typically ranging from 1B to 8B parameters, offer an alternative. Recent open-weight models such as Qwen 2.5 [18], Llama 3.2 [5], and Phi-4 [1] can run on consumer-grade hardware, enable local deployment, and drastically reduce per-query costs. However, out of the box, SLMs struggle with the core capabilities that agentic systems demand. They frequently produce malformed structured outputs, select incorrect tools, hallucinate function arguments, and fail at multi-step reasoning. These shortcomings have led to a widespread assumption that reliable agentic behavior requires large-scale models.

The failure mode is concrete. Consider a user query such as “*What is the sum of 265 and 345?*” directed at Qwen 2.5 7B-Instruct without any optimization. The available tool is an `add` function that accepts two numeric arguments. The expected output is a valid function call such as `{"name": "add", "arguments": {"a": 265, "b": 345}}`. Instead, the unoptimized model typically produces output of the form:

The sum of 265 and 345 is 610. You can calculate this using the add function with parameters a=265 and b=345.

The model has understood the query and identified the correct tool and arguments, but has wrapped the answer in conversational prose that fails to parse as JSON. On BFCL v4 `simple_python`, 394 out of 400 such responses from the unoptimized 7B model fail with a JSON parse error, yielding 1.5% overall accuracy. The problem is not that the model lacks the knowledge to answer; it is that the model cannot reliably express that knowledge in the structured format that agentic systems require.

This thesis challenges that assumption. Rather than treating model scale as the primary driver of agent reliability, it investigates whether targeted optimization techniques can bridge the performance gap between small and large models for agentic tasks. If small models can be made reliable enough for tool calling and reasoning through methods such as constrained decoding, prompt engineering, and fine-tuning, they could enable a new class of local, cost-efficient, and privacy-preserving agentic systems.

The practical deployment context motivating this investigation is the cascade architecture: a two-tier system in which a locally deployed small model handles the majority of incoming requests, while a frontier model accessed through an external API handles the remainder. This pattern is economically motivated. Frontier model APIs are charged per token; a locally deployed 7B model on a single consumer GPU incurs a fixed hardware cost independent of query volume. If the small model handles 75% of requests correctly, 75% of per-query API costs are eliminated. The central challenge is extending the fraction of

requests that the small model can handle reliably, which is precisely what each optimization technique in this thesis is evaluated against.

The optimization techniques are ordered by implementation cost. Methods that require no modification to the model weights – constrained decoding, prompt engineering, retrieval-augmented generation, and post-training quantization – are evaluated first, as they can be applied to any pre-trained checkpoint without additional compute. LoRA fine-tuning is evaluated last, as it requires a training run on a GPU cluster. This ordering reflects a practical decision ladder: a deployment team can adopt the no-training configurations immediately and defer the training investment until the marginal accuracy gain justifies it.

1.2 Problem Statement

Agentic systems built on language models must perform two fundamental capabilities well: reasoning correctly about user requests and calling the right tools with valid arguments. Reasoning requires the model to decompose tasks, decide when to call a tool versus respond directly, and chain multi-step logic without hallucinating intermediate steps. Tool calling requires selecting the correct function from a schema, producing valid structured arguments, and handling the result appropriately.

When small language models are used for these tasks without any optimization, they exhibit several failure modes. They generate invalid JSON, select non-existent tools, produce arguments with incorrect types or values, and fail to follow execution plans. These failures are not merely inconvenient; they render the agent non-functional for production use.

Two categories of failure are distinguishable. Format failures occur when the model produces syntactically invalid output: malformed JSON, unclosed brackets, incorrect field names, or free-form text where a structured object is required. These failures are independent of whether the model understood the query; they reflect an inability to express a correct answer in the required format. Semantic failures occur when the output is structurally valid but contains incorrect values: wrong function arguments, hallucinated parameters, or mismatched types. These failures reflect gaps in the model’s learned associations between query context and argument conventions.

The distinction matters because the two failure types respond to different interventions. Format failures can be eliminated at inference time by constrained decoding, which masks invalid tokens at each generation step without modifying the model. Semantic failures cannot be addressed by structural constraints; they require either training-based interventions that update the model’s learned priors, or retrieval-based methods that ground generation in retrieved factual content. The central question is therefore not whether SLMs fail, but which failure type dominates and which intervention addresses it most effectively.

The central question is whether these failures stem from fundamental limitations in model capacity or from insufficient constraints and guidance during generation. If the latter, then a carefully chosen stack of optimization techniques should be able to make small models competitive with frontier models on agentic benchmarks.

1.3 Research Questions

This thesis addresses three research questions:

RQ1 *What is the performance gap between unoptimized small language models (1B–8B parameters) and frontier models on tool-calling benchmarks?*

RQ2 *What is the marginal contribution of each optimization technique—constrained decoding, prompt engineering, retrieval-augmented generation, quantization, and LoRA fine-tuning—when applied cumulatively to small models for tool calling?*

RQ3 *Can a fully optimized small language model configuration achieve production-viable tool-calling accuracy on consumer hardware?*

RQ1 establishes the baseline gap that motivates the work. RQ2 is the core empirical contribution: a cumulative ablation study that isolates the marginal impact of each technique. RQ3 addresses the practical question of whether the optimized configuration is viable for real-world deployment.

To answer these questions, the thesis evaluates a cumulative pipeline of methods, ordered by implementation cost. No-training methods are evaluated first (constrained decoding [23], prompt engineering [3], inference-time compute [22], retrieval-augmented generation [13], quantization [14]), followed by training-based methods (LoRA fine-tuning [8]). Each method is assessed both individually and in combination, measuring its marginal contribution to tool-calling accuracy and reasoning quality.

1.4 Contributions

The main contributions of this thesis are:

1. A demonstration that constrained decoding alone recovers 71.25 percentage points of BFCL accuracy on Qwen 2.5 7B, lifting the unoptimized baseline from 1.5% to 72.75% by eliminating format failures entirely, without any modification to model weights.
2. A cumulative ablation study across ten configurations that isolates the marginal contribution of constrained decoding, prompt engineering, inference-time chain-of-thought, retrieval-augmented generation, quantization, and LoRA fine-tuning to BFCL `simple_python` accuracy. The best configuration (Config CD+FT-aligned, 76.75%) matches Claude Opus 4.5 (76.83%) on the same benchmark.
3. A model-size scaling analysis showing that constrained decoding is effective across the full Qwen 2.5 size range (0.5B to 7B), with CD gains ranging from 48.0 pp at 0.5B to 71.25 pp at 7B, and the 0.5B model already approaching GPT-5-mini (59.92%) under constrained decoding.
4. A characterization of the pipeline boundary: the two-stage single-call architecture achieves 0% accuracy on `multiple_function` and `parallel_function` categories, establishing where architectural changes rather than optimization techniques are required.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2 provides the theoretical background on language models, agentic systems, and the optimization techniques investigated in this work, including constrained decoding, prompt engineering, inference-time compute, retrieval-augmented generation, quantization, and LoRA fine-tuning.

Chapter 3 describes the experimental design, the cumulative evaluation ladder, the candidate models, and the evaluation framework including benchmarks and metrics.

Chapter 4 presents the experimental results for each optimization method, both individually and in combination, along with comparisons to frontier models.

Chapter 5 discusses the implications of the results, analyzes the cost-benefit trade-offs of each method, and addresses limitations of the study.

Chapter 6 answers the research question, summarizes the findings, and outlines directions for future work.

2 Background

This chapter introduces the theoretical foundations underlying this thesis. It covers language model architectures and their scaling properties, the design of agentic systems, and the optimization techniques evaluated in subsequent chapters.

2.1 Language Models

2.1.1 Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. [21], forms the foundation of modern language models. It replaces recurrent computation with a self-attention mechanism that allows each token in a sequence to attend to all other tokens in parallel. A Transformer block consists of a multi-head self-attention layer followed by a position-wise feed-forward network, with layer normalization and residual connections applied around each sub-layer.

Given an input sequence of token embeddings X , the self-attention mechanism computes queries $Q = XW^Q$, keys $K = XW^K$, and values $V = XW^V$ through learned linear projections. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \quad (2.1)$$

where d_k is the dimension of the key vectors. Multi-head attention runs this operation in parallel across multiple subspaces, allowing the model to capture different types of dependencies simultaneously.

For language modeling, the decoder-only variant is most common. It applies a causal attention mask so that each token can only attend to preceding tokens, enabling autoregressive generation where the model predicts one token at a time conditioned on all previous tokens.

Efficient autoregressive generation is supported by the key-value (KV) cache, which stores the key and value matrices computed for all previous tokens so that they do not need to be recomputed at each generation step. Without caching, generating a sequence of T tokens requires $O(T^2)$ attention operations; with caching, each new token requires only $O(T)$ operations. The KV cache grows linearly with sequence length and the number of attention heads, and its memory footprint becomes a significant constraint at long context lengths or large batch sizes.

Grouped-query attention (GQA) reduces KV cache memory by sharing a single set of key and value heads across multiple query heads. In standard multi-head attention, each of the h query heads has a corresponding key and value head, resulting in $2h$ matrices stored in the KV cache per layer. With GQA, h query heads are divided into g groups, each sharing one key and one value head, reducing the cache to $2g$ matrices per layer where $g \ll h$. GQA is used in the Qwen 2.5 model family [18] and is a key reason why the 7B model fits within consumer GPU memory budgets during inference.

2.1.2 Scaling Laws

Scaling laws describe the empirical relationship between model performance and three key factors: the number of parameters, the amount of training data, and the available compute budget. Kaplan et al. [10] showed that language model loss follows a power-law relationship with each of these factors, and that performance improves predictably as any of them increases.

Hoffmann et al. [7] refined these findings with the Chinchilla scaling laws, demonstrating that many large models were under-trained relative to their size. Their work showed that for a fixed compute budget, optimal performance is achieved by balancing model size and training data rather than maximizing parameters alone. This insight has direct implications for small language models: a well-trained small model can outperform a larger but under-trained one.

More recently, the focus has shifted toward inference-time scaling [20], where additional computation at generation time (through techniques such as chain-of-thought prompting or search) can improve performance without increasing model size.

2.1.3 Small vs. Large Language Models

Large language models (LLMs) such as GPT-4 [15] and the Claude model family operate at hundreds of billions of parameters and demonstrate strong performance across a wide range of tasks, including complex reasoning and tool use. However, their size demands significant computational resources, typically requiring multi-GPU clusters or cloud API access for inference.

Small language models (SLMs), in the range of 1B to 8B parameters, can run on consumer hardware such as a single GPU with 24GB of VRAM. Recent open-weight SLMs, including Qwen 2.5 [18], Llama 3.2 [5], and Phi-4 [1], have narrowed the gap with larger models on standard benchmarks through improved training data, better tokenizers, and architectural refinements such as grouped-query attention (GQA).

Despite these improvements, SLMs still lag behind frontier models on tasks that require multi-step reasoning, tool selection from large schemas, and reliable structured output generation. This gap motivates the optimization techniques explored in this thesis.

2.2 Agentic Systems

An agentic system is a software architecture in which a language model acts as the central reasoning engine, deciding which actions to take, invoking external tools, and synthesizing results to fulfill user requests. Unlike simple question-answering, agentic behavior involves planning, acting, and adapting based on intermediate results.

2.2.1 Tool Calling

Tool calling refers to the ability of a language model to invoke external functions or APIs as part of its response. The model receives a set of available tool schemas (names, descriptions, parameter types) and must select the appropriate tool for a given request, generate valid arguments matching the schema, and incorporate the tool’s response into its reasoning.

Tool calling is the primary capability evaluated in this thesis. It requires both language understanding (to map user intent to the correct tool) and structured generation (to produce syntactically and semantically valid function calls).

2.2.2 Structured Output Generation

Agentic systems typically require the model to produce outputs in a specific format, most commonly JSON. The model must generate output that is syntactically valid (parseable JSON), conforms to a predefined schema (correct field names and types), and is semantically correct (values that make sense for the given context).

Without explicit constraints, language models frequently produce malformed outputs: missing closing brackets, incorrect field names, wrong data types, or extra fields not present in the schema. These failures are particularly common in small models and represent a key reliability bottleneck for agentic deployments.

2.2.3 The ReAct Pattern

ReAct (Reasoning and Acting) [27] is a prompting framework that interleaves reasoning traces with action steps. Rather than generating a final answer directly, the model follows a loop of Thought (reasoning about what to do next), Action (calling a tool or performing a step), and Observation (processing the result of that action).

This pattern improves multi-step task completion by making the model’s reasoning explicit and allowing it to adapt its plan based on intermediate results. ReAct is particularly relevant for agentic systems because it provides a structured way to combine reasoning and tool use within a single generation process.

2.2.4 Cascade Architectures

A cascade architecture routes inference requests across two or more models of increasing capability and cost. Routine requests are handled by a smaller, faster model deployed locally; requests that exceed its reliable operating range are escalated to a larger model, typically accessed through an external API. The routing decision may be based on query complexity, output confidence, or post-hoc structural validation of the smaller model’s response.

The economic motivation is that frontier model APIs are charged per token and introduce network round-trip latency, while a locally deployed small model incurs a fixed hardware cost independent of query volume. If the small model handles a fraction p of requests correctly, the fraction of traffic escalated to the frontier model is at most $1 - p$, reducing variable inference cost in proportion.

For tool-calling tasks, structural validity is a necessary but not sufficient condition for accepting a small model’s response. A response that is syntactically valid JSON and names an existing function may still contain incorrect argument values. The residual semantic failure rate after structural constraints are enforced therefore determines the escalation boundary: queries whose failure mode is argument value errors cannot be filtered by structural checks alone.

This thesis is framed around extending the small model operating region within a cascade. Each optimization technique is evaluated by how much it reduces the residual failure rate, and correspondingly the fraction of traffic that must be escalated to a frontier model.

2.3 Constrained Decoding

Constrained decoding modifies the generation process so that the model can only produce outputs that conform to a predefined structure. Rather than relying on the model to implicitly learn output formats, constrained decoding enforces them at inference time by masking invalid tokens at each generation step.

2.3.1 Formal Grammars for Generation

The key idea is to define the valid output space using a formal grammar, typically a context-free grammar (CFG) or a JSON schema. At each decoding step, the grammar determines which tokens are valid continuations given the tokens generated so far. Invalid tokens receive a probability of zero (or negative infinity in log-space), ensuring that only grammatically valid sequences can be produced.

For tool calling, this means the model is guaranteed to produce valid JSON that conforms to the tool’s argument schema. Field names, types, and structure are enforced by construction rather than learned behavior. Willard and Louf [23] formalized this approach using finite-state machines compiled from regular expressions and JSON schemas.

2.3.2 Guided Decoding Techniques

Several libraries implement constrained decoding for language models. Outlines [24] uses finite-state machines compiled from regular expressions or JSON schemas to guide generation. LMQL [2] combines constrained decoding with a query language that allows expressing complex output constraints declaratively.

These techniques are particularly impactful for small models because they eliminate an entire class of errors (malformed outputs) without requiring any additional training. Constrained decoding is the foundational method in this thesis and serves as the base layer upon which other techniques are stacked.

2.4 Prompt Engineering and Inference-Time Compute

2.4.1 Few-Shot Prompting

Few-shot prompting provides the model with examples of correct input-output pairs directly in the prompt [3]. For tool calling, this means including examples of user requests paired with the expected function calls. The model uses these examples as templates to guide its own generation.

Few-shot prompting is a zero-cost technique (no training required) that can significantly improve task performance, especially for smaller models that benefit from explicit demonstrations of the expected output format and reasoning process.

2.4.2 Chain-of-Thought Reasoning

Chain-of-thought (CoT) prompting [22] encourages the model to generate intermediate reasoning steps before producing a final answer. Instead of directly outputting a tool call, the model first explains its reasoning: which tool is appropriate, what arguments are needed, and why.

CoT is a form of inference-time compute, trading additional generated tokens for improved accuracy. It is particularly effective for tasks that require multi-step reasoning, such as selecting among many similar tools or constructing complex function arguments. For small models, CoT can compensate for limited implicit reasoning capacity by making the reasoning process explicit.

2.5 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [13] augments the model’s input with relevant information retrieved from an external knowledge source at inference time. In the context of tool calling, RAG retrieves relevant tool schemas and documentation based on the user’s request, rather than including all available tools in every prompt.

This approach offers two benefits for small models. First, it reduces the prompt length by only including relevant tools, which is important given the limited context windows of smaller models. Second, it reduces hallucination by grounding the model’s generation in retrieved factual content rather than relying solely on parametric knowledge.

A typical RAG pipeline for tool calling involves encoding tool descriptions into a vector store, retrieving the top- k most relevant tools for a given query using semantic similarity, and injecting the retrieved schemas into the model’s prompt before generation.

The retrieval step depends on a dense vector index. Each tool description is encoded into a fixed-dimensional embedding vector by a sentence encoder, and all embeddings are stored in an index that supports approximate nearest-neighbour search. At query time, the user’s request is encoded by the same model, and the k tool embeddings with the highest cosine

similarity to the query embedding are retrieved. FAISS (Facebook AI Similarity Search) is a widely used library for this purpose, providing efficient exact and approximate search over large embedding collections [13].

The primary failure mode of RAG for tool calling is candidate disambiguation. When multiple tools have semantically similar descriptions (for example, `calculate_area`, `calculate_triangle_area`, and `calc_area_triangle`), the retrieval step returns all of them as high-similarity candidates, and the language model must then select the correct one. This selection task requires understanding subtle distinctions in tool semantics that may not be apparent from descriptions alone. The quality of retrieval is measured by $\text{recall}@k$: the fraction of queries for which the correct tool appears among the k retrieved candidates. High $\text{recall}@k$ is necessary but not sufficient for accurate tool calling; the model must still identify the correct tool from the retrieved set.

2.6 Parameter-Efficient Fine-Tuning

Fine-tuning adapts a pre-trained model to a specific task by continuing training on task-specific data. Full fine-tuning updates all model parameters, which is computationally expensive and requires significant memory. Parameter-efficient fine-tuning (PEFT) methods address this by updating only a small subset of parameters while keeping the rest frozen.

2.6.1 LoRA

Low-Rank Adaptation (LoRA) [8] is the most widely used PEFT method. Instead of updating the full weight matrices during fine-tuning, LoRA injects trainable low-rank decomposition matrices alongside the frozen pre-trained weights. For a weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA learns two smaller matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, where $r \ll \min(d, k)$, such that the adapted weight becomes $W + AB$.

This reduces the number of trainable parameters by orders of magnitude while achieving performance comparable to full fine-tuning on many tasks. For tool calling, LoRA can be used to fine-tune small models on datasets of function-calling examples, teaching them to select tools and generate arguments more accurately.

2.6.2 PEFT Variants

Beyond LoRA, several other PEFT methods exist. QLoRA [4] combines LoRA with quantization, applying low-rank adaptation on top of a 4-bit quantized base model to further reduce memory requirements. Adapter layers insert small trainable modules between frozen Transformer layers. Prefix tuning prepends trainable continuous vectors to the key and value sequences in each attention layer.

This thesis primarily uses LoRA due to its simplicity, strong empirical performance, and wide support across frameworks and model architectures.

2.7 Model Quantization

Quantization reduces the numerical precision of model weights from the 16-bit or 32-bit floating-point formats used during training to lower-bit integer representations at inference time. A weight stored as a 4-bit integer requires one quarter of the memory of a 16-bit floating-point value, enabling models that would otherwise require multi-GPU setups to run on a single consumer GPU. Secondary benefits include faster weight loading from storage and reduced memory bandwidth during inference.

Post-training quantization (PTQ) applies the precision reduction after training is complete, without additional gradient computation. This contrasts with quantization-aware training (QAT), which simulates quantization during training and achieves higher accuracy at the

cost of retraining. PTQ is preferred for large language models because the cost of retraining at scale is prohibitive; the challenge is to maintain accuracy despite the information loss from reduced precision.

2.7.1 Activation-Aware Weight Quantization

Naive PTQ at 4-bit precision produces significant accuracy degradation because not all weights tolerate uniform quantization equally. Weights connected to channels with large activation magnitudes have a disproportionate effect on output quality; quantizing these salient weights to 4 bits introduces large relative errors where the model is most sensitive.

Activation-aware Weight Quantization (AWQ) [14] identifies salient channels by examining activation statistics on a small calibration dataset and applies a per-channel scaling transformation that reduces the effective quantization error for high-magnitude channels. The weights are then quantized uniformly to INT4 after scaling. The transformation is absorbed into the preceding layer weights, so inference requires no additional computation beyond standard INT4 matrix multiplication.

For the Qwen 2.5 7B-Instruct model used in this thesis, AWQ INT4 quantization reduces model memory from 14.25 GiB to 5.20 GiB, a 63.5% reduction, and reduces model loading time from 212 s to 35 s. The accuracy cost on BFCL v4 `simple_python` is -0.5 percentage points relative to the bfloat16 baseline, within run-to-run variance. Pre-quantized AWQ INT4 weights are published on the Hugging Face Hub for all Qwen 2.5 Instruct sizes [18], avoiding the need for manual quantization.

2.8 Related Work

2.8.1 Tool Calling and Benchmarks

The Berkeley Function Calling Leaderboard (BFCL) [25] provides a standardized benchmark for evaluating language models on function-calling tasks. Four categories of increasing difficulty are defined. In the `simple_python` category, a single oracle function definition is provided alongside the user query, and the model must produce the correct function call with matching arguments. In the `multiple_function` category, several candidate function definitions are provided and the model must first identify the correct function before extracting arguments. The `parallel_function` category requires multiple simultaneous function calls from a single user request. A multi-turn category tests stateful interactions across several dialogue turns, where argument values may depend on prior tool outputs. This thesis evaluates all configurations on the `simple_python` category, which isolates argument extraction accuracy from the function selection problem.

Two evaluation metrics are reported in BFCL. AST accuracy parses the model output into an abstract syntax tree and compares it structurally against the ground truth, checking function name, argument names, argument types, and argument values without executing the function. Execution accuracy runs the generated call and compares the runtime result to a ground truth output. AST accuracy is the primary metric used in this thesis because it is deterministic, reproducible, and independent of runtime environment, consistent with the leaderboard convention for comparing configurations systematically.

ToolLLM [17] extends function-calling evaluation to over 16,000 real-world APIs from the RapidAPI catalogue, covering domains including finance, weather, social media, and e-commerce. At this scale, the model must select among thousands of APIs with varying documentation quality and compose multi-step tool chains. The primary metric is execution accuracy, requiring correct runtime results rather than structural matching. The scope of ToolLLM reflects a harder generalization challenge than BFCL, as the API catalog far exceeds what can be memorized from pre-training data.

The τ -bench benchmark [26] evaluates end-to-end multi-step agentic task completion in simulated retail and airline customer service domains. Each task requires the model to reason across five to fifteen tool calls, observe intermediate results, and reach a final database state that matches a ground truth resolution. Task reward is binary: the final state must exactly match the ground truth to receive a score of 1.0, and partial completion yields no reward. A user simulator introduces stochastic variation across runs, so multiple evaluation passes are required for stable pass-rate estimates.

2.8.2 Fine-Tuned Small Models for Tool Use

Gorilla [16] demonstrated that a LLaMA model fine-tuned on API documentation can outperform GPT-4 on API calling tasks. The key finding is that training data relevance can compensate for parameter count disadvantage when the task requires structured generation against specific schemas. Gorilla uses a retrieval-augmented training procedure in which API documentation is retrieved during training, teaching the model to use retrieved context rather than rely on memorized API signatures.

The xLAM model family [28] extends the fine-tuning approach with a large-scale training pipeline that draws from diverse function-calling datasets. The `xlam-function-calling-60k` dataset used in this thesis for LoRA fine-tuning contains 60,000 function-calling examples across a wide range of API types and domains. The diversity is intended to improve generalization beyond the specific schemas seen during training. Models trained on this dataset are competitive with frontier systems on standard tool-use benchmarks.

The format of training outputs has a direct effect on how fine-tuning interacts with constrained decoding at inference time. This interaction is observed in the present experiments: two LoRA adapters trained on the same dataset with identical hyperparameters but different output formats converge to similar final training loss (0.203 and 0.229 at step 6,750 respectively) yet differ by 7.0 percentage points in BFCL accuracy when paired with the constrained decoding pipeline. The accuracy difference is attributable entirely to whether the training output format matches the inference-time generation target, not to any difference in training quality. This result extends Gorilla’s data-relevance insight from API source selection to output format alignment.

A recent survey of small language models for agentic systems [19] identifies Qwen 2.5 7B as one of the strongest open-weight models for tool use at sub-10B parameter scales, attributing its performance to instruction tuning on function-calling examples and the scale of its pre-training corpus. The survey notes that most existing evaluations test individual optimization techniques in isolation, and that systematic combinations of constrained decoding, quantization, and fine-tuning remain underexplored. This gap directly motivates the cumulative evaluation design adopted in this thesis.

2.8.3 Constrained Generation

Willard and Louf [23] formalized constrained generation using finite-state machines (FSMs) compiled from regular expressions and JSON schemas. At each decoding step, the current grammar state determines the set of valid next tokens; tokens outside this set receive a logit of negative infinity before sampling, ensuring that the generated sequence conforms to the grammar by construction. The key contribution is an efficient offline compilation algorithm that converts JSON schemas into FSMs before inference begins, so that the per-step cost at generation time consists only of a state transition lookup.

The Outlines library [24] implements the FSM approach and serves as the constrained generation backend used in this thesis via vLLM [11]. Two generation modes are relevant to the two-stage tool-calling pipeline. The `guided_choice` mode restricts output to one

string from a predefined list and is used in Stage 1 to guarantee that the predicted function name exists in the available schema. The `guided_json` mode restricts output to a JSON object conforming to a provided schema and is used in Stage 2 to guarantee structural validity of the argument object. Both modes compile their constraint to a token mask at request initialization time, amortizing the compilation cost across all generation steps for a given request.

The per-step overhead of FSM-based constrained generation is low when schemas are compiled ahead of time. Overhead scales with schema complexity: schemas with deeply nested structures and many optional fields require larger FSM state spaces and more expensive state transitions. For the function schemas in BFCL `simple_python`, complexity is moderate and the overhead falls within practical inference budgets. Small models benefit disproportionately from constrained generation because their baseline format error rates are higher: without constraints, Qwen 2.5 7B produces unparseable output on 98.5% of BFCL test cases, a failure rate that constrained decoding eliminates entirely.

LMQL [2] offers an alternative approach through a declarative query language in which output constraints are expressed as Python predicates evaluated during generation. The approach is more flexible than FSM-based methods for constraints that are difficult to specify as formal grammars. The per-step cost is higher, however, because predicates are evaluated at each generation step rather than looked up from a precompiled state table. This thesis uses FSM-based constrained decoding via vLLM because it is better integrated with batch inference and provides formal guarantees from a compiled grammar state.

2.8.4 Cascade Architectures

A cascade architecture routes inference requests across models of different capability and cost [20]. Routine requests are handled by a smaller, faster model; requests that exceed its reliable operating range are escalated to a larger model, typically accessed through an external API. Routing decisions may be based on output confidence, output validation, or a query complexity estimate computed before inference.

For tool-calling tasks, structural validation is a natural routing gate. If the small model output fails to parse as valid JSON or names a function not present in the available schema, the response can be rejected and escalated without execution. Constrained decoding removes this failure mode by construction, since every output is structurally valid by guarantee. After constrained decoding is applied, the residual failure rate is attributable entirely to incorrect argument values, which structural checks cannot detect. This shifts the routing problem from format detection to semantic verification, a harder problem that requires either ground-truth access or execution feedback.

The residual semantic failure rate determines the fraction of traffic that must be escalated to a frontier model and the associated variable inference cost. Config CD achieves 72.75% accuracy, implying a 27.25% escalation rate under structural routing. Config CD+FT-aligned reduces this to 23.25%, a reduction of approximately 4.0 percentage points. The relationship between optimization technique selection and cascade routing cost is a practical design consideration that has received limited attention in prior work; this thesis provides direct measurements for five technique combinations.

2.8.5 Small Model Optimization

Recent model families have narrowed the capability gap with frontier models through improvements applied at training time. Phi-4 [1] achieves competitive performance with models several times its size on reasoning and mathematics benchmarks, demonstrating that training data curation can compensate for reduced parameter count. Llama 3 [5]

introduces grouped-query attention and an expanded vocabulary to improve inference efficiency and multilingual coverage. Qwen 2.5 [18] achieves strong tool-use performance through instruction tuning on function-calling examples and a context window of 128,000 tokens.

Knowledge distillation [6] provides a complementary path to capable small models. A small student model is trained to match the output distribution of a large teacher rather than ground-truth labels, transferring reasoning capabilities that the student would not acquire from labeled data alone. Unlike the post-training techniques evaluated in this thesis, distillation affects the model’s parametric knowledge and requires access to the teacher model’s outputs during training.

Quantization reduces model memory at the cost of a small accuracy penalty. For the Qwen 2.5 7B model used in this thesis, AWQ INT4 quantization [14] reduces memory from 14.25 GiB to 5.20 GiB and reduces accuracy by 0.5 percentage points on `BFCL simple_python`, a cost within run-to-run variance. This result is consistent with reported accuracy retention for AWQ across other model families at similar parameter scales.

The pre-training and instruction-tuning improvements embodied in Phi-4, Qwen 2.5, and Llama 3 are largely orthogonal to the inference-time and post-training techniques evaluated here. Constrained decoding, quantization, and LoRA fine-tuning can be applied on top of any pre-trained checkpoint. The question addressed in this thesis is how much additional accuracy is recovered by combining these techniques on top of an already well-trained small model, and which combinations offer the best accuracy-cost tradeoff for deployment in a cascade architecture.

3 Methodology

The experiments are structured around a simple logic: a bare baseline is defined first, and one technique is added at a time. The Qwen 2.5 model family is used, tested across four sizes from 0.5B to 7B. Five techniques are covered: constrained decoding, quantization, few-shot prompting, RAG, and LoRA fine-tuning. All configurations are evaluated on the Berkeley Function Calling Leaderboard.

3.1 Experimental Design

The experimental design is based on controlled comparisons. A baseline configuration is established first, and each technique is added one at a time. This isolates the effect of each technique on tool-calling accuracy.

3.1.1 Cumulative Evaluation Ladder

Five configurations are evaluated, each building on the previous one. Config B is the baseline: the model is prompted without constrained decoding at full precision. In Config CD, constrained decoding is added, forcing the output to conform to a valid JSON schema. In Config CD+Q, AWQ INT4 quantization is applied to reduce memory footprint while preserving accuracy as much as possible. In Config CD+Q+FT, LoRA fine-tuning is added to the quantized model. Two parallel configurations are also evaluated: Config PE uses few-shot prompting, and Config CD+Q+RAG adds retrieval-augmented generation to Config CD+Q.

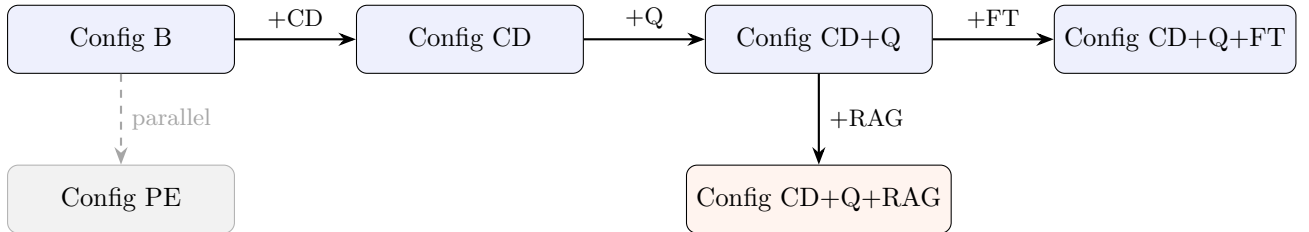


Figure 3.1: Cumulative evaluation ladder. Each configuration in the main chain (top row) extends the previous one by adding a single technique. Config CD+Q+RAG and Config PE are evaluated as parallel tracks.

3.1.2 Hardware and Infrastructure

All experiments are conducted on the DTU HPC cluster (gbar.dtu.dk), running AlmaLinux 9.7 under the LSF job scheduler. Each job is allocated one NVIDIA A100 80 GB PCIe GPU in exclusive process mode, eight CPU cores, and 64 GB of RAM. Inference jobs use a two-hour wall-time limit; LoRA training jobs use three hours for models up to 1.5B parameters and eight hours for the 7B model.

Model inference is served via vLLM 0.8.5, which exposes a REST API on localhost that the evaluation client connects to, with GPU memory utilisation limited to 90%. Full-precision models are loaded in bfloat16 (14 GB VRAM for the 7B model); quantized models are loaded in float16 under the AWQ Marlin backend (4–5 GB VRAM). The software environment uses uv 0.5.13, Python 3.12.11, and CUDA 12.6.3. Model weights are cached from the Hugging Face Hub on cluster storage.

3.2 Evaluation Framework

Two benchmarks are used to evaluate the configurations. The Berkeley Function Calling Leaderboard covers single-turn tool-calling accuracy across categories of increasing complexity. τ -bench covers multi-step agentic task completion, where the model must reason, call tools, observe results, and iterate until a task is done.

3.2.1 Berkeley Function Calling Leaderboard

All configurations are evaluated on the Berkeley Function Calling Leaderboard (BFCL) v4 [25]. Three categories are used: `simple_python` (400 test cases, one candidate function per test), `multiple_function` (the model must select the correct function from several candidates), and `parallel_function` (the model must produce multiple function calls simultaneously). Each test case consists of a user prompt and one or more function definitions; the model is required to produce the correct function call with the correct arguments. Frontier model scores are taken from the published BFCL leaderboard rather than re-evaluated, as the official scores are the canonical reference for the benchmark.

The primary metric is AST accuracy: the model output is parsed and compared structurally against the ground truth, checking function name, argument names, argument values, and argument types. Evaluation is performed using the BFCL `ast_checker`, which applies lenient type coercion between compatible types (e.g., integer and float). All inference runs use temperature 0 to ensure deterministic outputs across configurations. Function selection accuracy and argument extraction accuracy are reported separately to identify where failures occur along the prediction pipeline.

3.2.2 τ -bench

The τ -bench retail benchmark [26] evaluates end-to-end multi-step task completion. Each task is a simulated customer service interaction in which the model must reason across 5–15 tool calls, observe intermediate results, and reach a final database state that matches a ground truth resolution. Config CD is evaluated on the retail domain test split (115 tasks) to establish the agentic baseline; the remaining configurations are not evaluated on τ -bench because the primary bottleneck identified is multi-turn planning rather than single-call argument accuracy.

Three full evaluation runs are completed to account for variance introduced by the user simulator. The user simulator is Qwen 2.5 7B-Instruct served on the same vLLM instance as the agent model, running at temperature 0.3 to produce naturalistic user responses. The agent model runs at temperature 0.0 to ensure deterministic tool call generation. All runs are executed on a single NVIDIA L40S GPU (48 GiB) on the DTU HPC cluster.

Scoring is binary: a task receives reward 1.0 only if the final database state exactly matches the ground truth at the end of the interaction, and reward 0.0 otherwise. Partial task completion yields no reward. The pass rate is reported as the mean reward across all tasks and all runs.

3.3 Models

3.3.1 Qwen 2.5

The Qwen 2.5 model family [18] is used across all experiments. Four sizes are evaluated: 0.5B, 1.5B, 3B, and 7B parameters, all in the Instruct variant. The 7B model is the primary subject of analysis; the smaller sizes are included to study how model scale interacts with each optimization technique.

Qwen 2.5 is selected for three reasons. First, the family covers a continuous size range under a single architecture and training regime, which allows scale effects to be isolated

without confounding differences in model design. Second, native AWQ INT4 quantized variants are available for all sizes on the Hugging Face Hub, avoiding the need for manual quantization. Third, recent surveys of SLMs for agentic systems identify Qwen 2.5 7B as one of the strongest open-weight models for tool use and function calling at this parameter scale [19]. All models are served under an Apache 2.0 licence.

All Qwen 2.5 models share a decoder-only Transformer architecture with grouped-query attention (GQA), SwiGLU feed-forward activations, and rotary position embeddings (RoPE). The 7B variant uses 28 layers, a hidden dimension of 3584, and 28 query heads grouped into 4 key-value head groups. The supported context length is 128,000 tokens, though all experiments in this thesis operate well within this limit. GQA reduces the KV cache memory footprint relative to standard multi-head attention, which is one reason the 7B model fits comfortably on a single consumer GPU in bfloat16 precision.

The Instruct variant is pre-trained on a multilingual corpus of approximately 18 trillion tokens, followed by supervised fine-tuning on instruction-following and tool-use examples, and alignment via reinforcement learning from human feedback. The resulting model responds to structured prompts and tool schemas without additional task-specific training, making it a practical starting point for the optimization ladder evaluated in this thesis.

3.4 Baseline Definition

Config B is the baseline configuration: the model is prompted without constrained decoding, at full precision (bfloat16), with no quantization, no fine-tuning, and no few-shot examples. The pipeline is two-stage (Figure 3.2). In the first stage, the available function signatures and descriptions are presented to the model, and the prompt ends with **Function name:** ; the model completes freely. In the second stage, the selected function signature and the user query are presented, and the prompt ends with **JSON:** ; the model generates argument values as free-form text. A JSON extraction heuristic is applied to the output, capturing the substring from the first opening brace to the last closing brace. Temperature is set to 0 across all configurations to ensure deterministic outputs, removing randomness as a confound when comparing techniques.

3.5 Constrained Decoding

Config CD adds constrained decoding to Config B; all other settings remain unchanged. Constrained decoding is implemented via vLLM’s guided decoding feature [11, 23], which applies logit masking at each generation step to restrict the output to a valid token sequence.

Two mechanisms are used in the two-stage pipeline. In the function selection stage, `guided_choice` restricts the output to one of the valid function names, guaranteeing that the selected name exists in the available function set. In the argument extraction stage, `guided_json` restricts the output to a JSON object that conforms to a schema derived from the function definition, with parameter names, types, and required fields enforced, and `additionalProperties` set to false.

3.6 Quantization

Config CD+Q adds AWQ INT4 quantization [14] to Config CD. The pre-quantized model `Qwen/Qwen2.5-7B-Instruct-AWQ` is used, loaded via vLLM with the `awq_marlin` backend, which applies optimized Marlin kernels for efficient INT4 inference on GPU. All other settings remain unchanged.

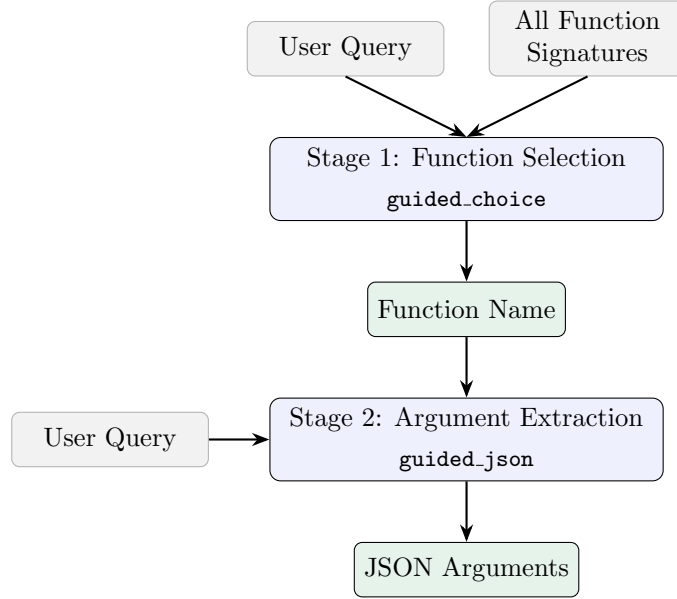


Figure 3.2: Two-stage inference pipeline. Stage 1 selects a function name from all available candidates; Stage 2 extracts argument values for the selected function. In Config CD and above, `guided_choice` and `guided_json` enforce structural validity at each stage.

AWQ is selected because activation-aware weight quantization preserves accuracy better than uniform quantization by protecting weights that correspond to large activation values. Pre-quantized variants for all Qwen 2.5 sizes are available on the Hugging Face Hub, avoiding the need for a local quantization step.

3.7 Prompt Engineering and Few-Shot

Config PE extends Config CD by prepending three few-shot examples to the argument extraction prompt. The function selection prompt is left unchanged; examples are added only to the argument extraction stage, where all Config CD failures occur.

The three examples are constructed to target the most frequent failure categories observed in Config CD. The first example demonstrates numeric precision: the expected value is 9.81, not the rounded approximation 9.8. The second demonstrates string format with a location qualifier: the expected value is "San Francisco, CA", not "San Francisco". The third demonstrates Python expression syntax: the expected value is `x**2`, not the mathematical notation x^2 . Three examples are selected as the minimum number needed to cover distinct failure classes; additional examples would increase context length without introducing new failure categories.

All other settings remain unchanged from Config CD: the full-precision Qwen 2.5 7B-Instruct model is used, guided decoding is applied to both pipeline stages, and temperature is set to 0. Config PE is evaluated as a parallel track to Config CD+Q rather than as an extension of it, so the full-precision model is retained to isolate the effect of the few-shot examples from any interaction with quantization.

3.8 Retrieval-Augmented Generation

Config CD+Q+RAG extends Config CD+Q by replacing the per-test-case oracle function provision with a retrieval step [13]. In all prior configurations, each test case is presented with the exact function definition required to answer it. In Config CD+Q+RAG, the model

receives instead the top- k functions retrieved from an index of the full function catalogue.

All 370 unique function signatures from the `simple_python` category are embedded using `sentence-transformers/all-MiniLM-L6-v2` and indexed with FAISS before evaluation begins. The `all-MiniLM-L6-v2` model is selected because it is designed for semantic similarity of short texts, requires no GPU, and is approximately 80 MB, making it compatible with the vLLM server that holds the GPU in exclusive process mode. The embedder is run on CPU to avoid a device ownership conflict with vLLM.

At inference time, the user query is embedded and the top-5 most similar functions are retrieved as candidates ($k = 5$). The value $k = 5$ is selected to achieve near-complete recall while keeping the candidate set small enough for the model to disambiguate: a smaller k risks excluding the correct function, while a larger k increases the number of semantically similar alternatives the model must distinguish. The model selects among the retrieved candidates using `guided_choice` restricted to the five retrieved function names, then extracts arguments using `guided_json` as in Config CD+Q. All other settings are inherited from Config CD+Q.

3.9 LoRA Fine-Tuning

Two fine-tuning configurations are evaluated: Config FT-only, which applies the trained model without constrained decoding, and Config CD+FT, which combines the trained model with the constrained decoding pipeline from Config CD. The split between the two variants allows the independent contributions of fine-tuning and constrained decoding to be measured separately.

The base model is `Qwen/Qwen2.5-7B-Instruct` in bfloat16. A LoRA adapter is trained for 2 epochs with rank 16 on the Salesforce `xlam-function-calling-60k` dataset [28], using 54,000 training examples and 6,000 validation examples, with the HuggingFace PEFT library and TRL SFTTrainer [8].

Rank 16 is selected as a mid-range value for instruction-following adaptation: it provides sufficient capacity for the model to learn format and value conventions from the training data without overfitting. Two epochs are selected because the dataset is large enough that a single pass provides substantial exposure to the training distribution; a third epoch produced no further improvement in validation loss in preliminary runs.

After training, the adapter weights are merged into the base model by linear interpolation, producing a single merged checkpoint in bfloat16. The merged model is used for all evaluation runs, removing adapter inference overhead and ensuring that the evaluation infrastructure is identical to that used for Config B and Config CD.

4 Results

4.1 Baseline Performance

Table 4.1 summarizes the baseline results for Qwen 2.5 7B-Instruct on BFCL v4 simple_python (400 test cases).

Table 4.1: BFCL v4 Simple Function AST accuracy for Qwen 2.5 7B-Instruct across initial configurations.

Config	Description	Correct	Accuracy
B	Raw model (no optimization)	6/400	1.5%
PE	Few-shot prompting + guided	281/400	70.25%
CD	Constrained decoding (guided)	291/400	72.75%

Without constrained decoding (Config B), the model achieves only 1.5% accuracy. The dominant failure mode is format non-compliance: 394 out of 400 outputs fail to parse as valid JSON, producing **Extra data** errors or empty results. The model is not incapable of understanding the queries—it simply cannot express its answers in the required structured format.

Constrained decoding (Config CD) recovers 71.25 percentage points, achieving 72.75% accuracy. Function selection reaches 100% accuracy via **guided_choice**, confirming that the format enforcement fully solves tool selection. The remaining 27.25% failure rate is entirely in argument extraction: wrong numeric precision (e.g., 9.8 vs. 9.81), string format mismatches (e.g., “kilometers/hour” vs. “km/h”), and incorrect optional parameter defaults.

Few-shot prompting (Config PE) decreases accuracy by 2.5 percentage points relative to Config CD. The three few-shot examples, targeting known failure modes, did not transfer across function domains. This negative result indicates that the argument extraction errors reflect the model’s learned associations rather than a formatting gap that in-context examples can bridge.

4.2 No-Training Methods

The following configurations modify only the inference pipeline; model weights are unchanged in all cases. Configs CD and CD+Q establish the no-training ceiling. Configs PE, CD+Q+ITC, and CD+Q+RAG assess whether inference-time interventions can improve on that ceiling.

4.2.1 Constrained Decoding

The two-stage constrained decoding pipeline eliminates the dominant failure mode of Config B. In the function selection stage, **guided_choice** restricts generation to the set of available function names: function selection accuracy is 100% across all 400 test cases, confirmed by inspection of the raw prediction files. In the argument extraction stage, **guided_json** restricts generation to a JSON object conforming to the function schema, ensuring that every output is structurally valid [23, 11].

The 27.25% failure rate that remains after constrained decoding is entirely in argument values. Four categories account for the observed failures: wrong optional parameter

defaults (e.g., `root_type=''` where `'all'` is expected), numeric precision errors (e.g., 9.8 where 9.81 is expected), string format mismatches (e.g., `"kilometers/hour"` where `"km/h"` is expected), and location format errors (e.g., `"New York"` where `"New York, NY"` is expected). These failures share a common structure: the model’s output is semantically reasonable but does not match the specific convention that the benchmark’s ground truth encodes. Constrained decoding enforces structural validity but cannot enforce semantic convention.

AWQ INT4 quantization reduces model memory from 14.25 GiB to 5.20 GiB (63.5% reduction) and inference startup time from 212 s to 35 s (83.5% reduction), with an accuracy change of -0.5 pp (289/400, 72.25%) [14]. The failure distribution does not change: no new error categories are introduced, and the relative frequency of existing failure types is unchanged. The accuracy difference of two test cases falls within run-to-run variance, confirmed by six re-runs that consistently produced 289/400. Config CD+Q therefore provides equivalent accuracy to Config CD at a substantially reduced memory footprint, and serves as the operating baseline for subsequent no-training experiments. Fig. 4.1 plots GPU memory against accuracy for the main configurations.

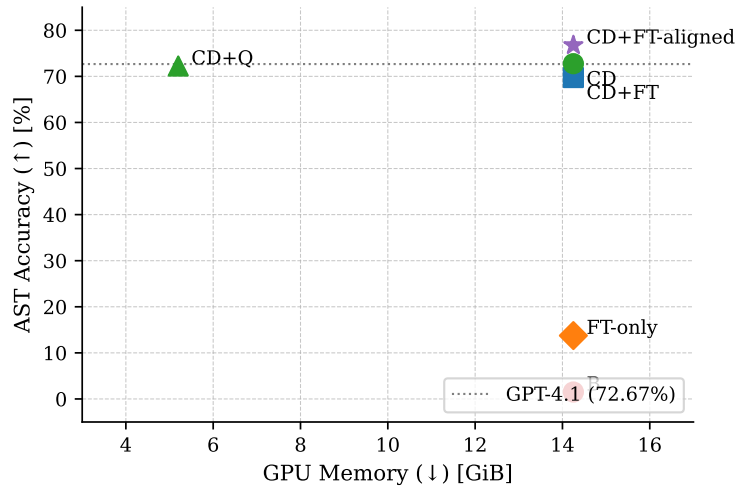


Figure 4.1: GPU memory footprint vs. BFCL simple_python AST accuracy for selected configurations. Config CD+Q achieves equivalent accuracy to Config CD at 63.5% lower memory. The GPT-4.1 reference line (72.67%) is shown for comparison.

4.2.2 Prompt Engineering and Few-Shot

Three few-shot examples are prepended to the argument extraction prompt, constructed to target the most frequent failure categories observed in Config CD: numeric precision (9.81 rather than 9.8), string format with a state qualifier (`"San Francisco, CA"`), and Python expression syntax (`x**2` rather than `x^2`). The result is 70.25% accuracy (281/400), a decrease of 2.5 pp relative to Config CD.

The failure categories from Config CD persist at similar or higher rates despite the targeted examples. The model does not transfer the demonstrated conventions to test cases from different function domains. Two mechanisms account for this outcome. First, the few-shot examples use function signatures from domains that differ from the test cases; the convention “use exact numeric precision” demonstrated for one function does not generalise to others. Second, the examples add approximately 300 tokens to each prompt without a proportional accuracy gain; for borderline cases where the base model produced a marginally correct output, the additional context appears to reduce accuracy slightly.

The result indicates that the argument extraction errors remaining after constrained decoding are not format gaps addressable by in-context examples [3]. The model’s internal representation associates “acceleration due to gravity” with 9.8 and “fuel type” with “gasoline”; these priors are not shifted by example presentation alone.

4.2.3 Inference-Time Compute

A chain-of-thought reasoning step is inserted between function selection and argument extraction in Config CD+Q+ITC [22]. After the correct function is selected by `guided_choice`, the model generates a free-form reasoning trace of up to 256 tokens before the final `guided_json` extraction. The result is 65.5% accuracy (262/400), a decrease of 6.75 pp relative to Config CD+Q, and the lowest accuracy among all configurations that use guided decoding.

A flip analysis, computed by comparing per-case outcomes between Config CD+Q and Config CD+Q+ITC, separates the net change into individual gains and losses. Of the 400 test cases, chain-of-thought recovered 24 cases that were incorrect under Config CD+Q, while breaking 51 cases that were correct. The net change is -27 cases. Fig. 4.2 shows the four-way breakdown.

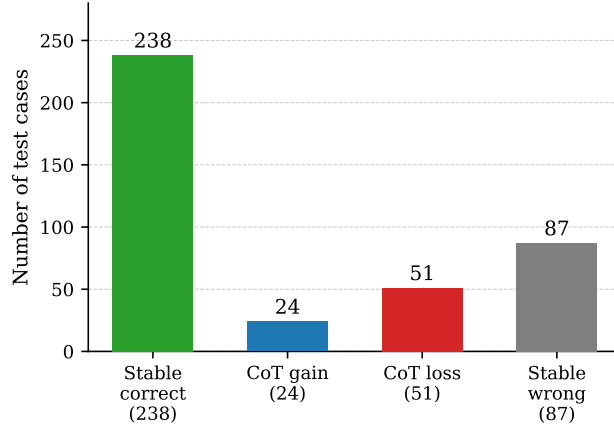


Figure 4.2: Flip analysis for Config CD+Q+ITC relative to Config CD+Q. Of 400 test cases, chain-of-thought breaks 50 previously correct predictions while recovering only 24 previously incorrect ones, for a net change of -26 cases.

Table 4.2 illustrates four representative losses. In each case, Config CD+Q produces the value accepted by the benchmark, and the reasoning step overrides it with a linguistically natural alternative that the benchmark does not accept.

Three mechanisms account for this pattern. First, the reasoning step introduces drift: the model interprets the instruction to reason step by step as licence to produce derivations and justification paragraphs before the extraction step, consuming tokens without supplying useful information. Second, values committed in the reasoning text act as a prior for the subsequent guided extraction: once the reasoning block contains `discount_rate = 10`, the guided step inherits that value rather than re-evaluating it. Third, the model normalises values toward linguistically standard forms. The benchmark’s ground truth encodes specific conventions, often arbitrary ones (abbreviations over full words, decimal fractions for percentages, particular date formats), that reasoning moves the model away from rather than toward. Config CD+Q’s one-step guided decode produces the expected convention without verbalising it; verbalising the reasoning surfaces the model’s preferred form, which consistently differs from the benchmark’s.

Table 4.2: Representative cases where chain-of-thought reasoning overrides a correct Config CD+Q prediction. Reasoning excerpts are taken verbatim from stored reasoning traces.

Parameter	CD+Q	CoT	Reasoning excerpt
<code>discount_rate</code>	0.1 ✓	10.0 ×	“The discount rate is in percent-age form. Extracted arguments: <code>discount_rate = 10</code> ”
<code>fuel_type</code>	‘gas’ ✓	‘gasoline’ ×	“Gasoline would be the fuel type”
<code>duration</code>	‘2 weeks’ ✓	‘last 2 weeks’ ×	“The duration is ‘last 2 weeks’, which needs to be preserved exactly as a string”
<code>start_date</code>	‘01/01/2021’ ✓	‘2021-01-01’ ×	“start_date: ‘2021-01-01’ (assuming ISO 8601 format)”

The result extends the finding from Config PE: the residual failure rate after constrained decoding is caused by a mismatch between the model’s priors and the benchmark’s labels, and interventions that make reasoning explicit exacerbate this mismatch.

4.2.4 Retrieval-Augmented Generation

In all preceding configurations, each test case is provided with the exact function definition required to answer it. Config CD+Q+RAG replaces this oracle provision with a retrieval step. All 370 unique function signatures from the `simple.python` category are embedded using `sentence-transformers/all-MiniLM-L6-v2` and indexed with FAISS. At inference time, the query is embedded and the top-5 most similar functions are retrieved as candidates; the model must select the correct function from these candidates and extract its arguments [13]. The result is 47.75% accuracy (191/400), a decrease of 24 pp relative to Config CD+Q.

The retrieval component achieves recall@5 of 97.2% (389/400): the correct function is present among the 5 retrieved candidates in 389 of 400 test cases. The 11 retrieval failures occur on domain-ambiguous functions whose names share little lexical overlap with the query (e.g., `calculate_density`, `recipe_search`, `sentiment_analysis`). Retrieval quality is therefore not the primary cause of the accuracy decrease.

The failure distribution identifies where accuracy is lost. Of the 400 test cases, 264 (66.0%) produce output identical to Config CD+Q, indicating that the additional candidates have no effect on these cases. In 128 cases (32.0%), the model selects a wrong function from the retrieved candidates. In 8 cases (2.0%), the correct function is selected but with incorrect argument values. The dominant failure mode is function disambiguation. When a query for triangle area retrieves `calculate_triangle_area`, `calculate_area`, and `calc_area_triangle` as candidates, the model selects a generic variant in the majority of cases. Similar patterns are observed for mathematical synonyms (`algebra.quadratic_roots` vs. `solve_quadratic_equation`) and namespace variants (`geometry.circumference` vs. `calculate_circumference`).

The result establishes that the bottleneck in a RAG-based tool-calling pipeline for a 7B model is not retrieval quality but candidate discrimination. A recall@5 of 97.2% is sufficient for the retrieval component; the model lacks the capacity to reliably distinguish among semantically similar candidate functions once they are retrieved. Fig. 4.3 summarises the

failure distribution across the 209 incorrect predictions.

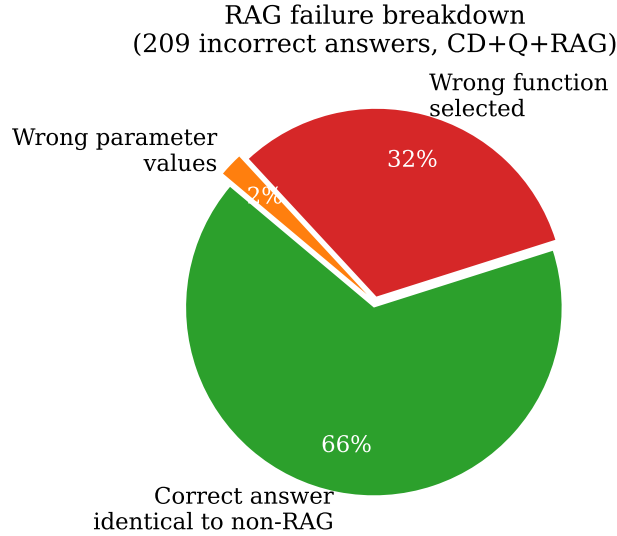


Figure 4.3: Failure breakdown for Config CD+Q+RAG (209 incorrect predictions out of 400 test cases). The dominant failure mode is wrong function selection (32%), not retrieval miss or argument extraction error.

4.3 Training-Based Methods

4.3.1 LoRA Fine-Tuning

A LoRA adapter is trained on the Salesforce `xlam-function-calling-60k` dataset [28], using 54,000 training examples and 6,000 validation examples. The base model is `Qwen/Qwen2.5-7B-Instruct`, trained for 2 epochs with LoRA rank 16 in `bfloat16` [8]. The adapter is merged into the base weights by linear interpolation before evaluation, producing a single merged model at full `bfloat16` precision.

Without constrained decoding (Config FT-only), the fine-tuned model achieves 13.75% accuracy (55/400), an improvement of 12.25 pp over the unguided baseline Config B (1.5%). Fine-tuning on a function-calling corpus teaches the model that tool-calling responses should be structured: parseable, single-object JSON is produced in more cases than by the base model. The 12.25 pp gain confirms that the training signal is meaningful and that the `xlam` examples contain sufficient format regularity for format learning to occur.

The `xlam` dataset encodes assistant-turn responses as Python call syntax (`func(arg=value, ...)`). This convention differs from the evaluation pipeline, which expects a JSON argument object (`{"arg": value}`). The format mismatch between the training distribution and the evaluation format is the primary limitation of this configuration, and its effect on the combined pipeline is examined in Section 4.4.

4.4 Combined Configurations

Config CD+FT combines constrained decoding with the LoRA-merged model. The result is 69.75% accuracy (279/400), a regression of 3 pp relative to Config CD without fine-tuning. The combined configuration underperforms the no-fine-tuning baseline.

The regression is attributed to the training format mismatch. Constrained decoding enforces the surface format: every output is a valid JSON object conforming to the function schema. However, the argument value predictions remain biased toward the conventions of

the training data. Since xlam encodes argument values as part of Python call syntax, the fine-tuned model’s weights encode this convention. When constrained decoding forces the output to be valid JSON, the value-level predictions are pulled toward xlam conventions rather than the BFCL ground truth. A secondary contributing factor is the composition of the training set: 52.6% of xlam examples contain multiple function calls per query, while the `simple_python` category is exclusively single-call. This distribution mismatch may shift the model’s prior toward multi-call patterns in contexts where a single call is required.

The gap between Config FT-only (13.75%) and Config CD+FT (69.75%) confirms that constrained decoding remains the dominant contributor to accuracy even after fine-tuning: removing it reduces accuracy by 56 pp. The two techniques are complementary, but fine-tuning on a misaligned training distribution does not improve on the CD baseline because the learned value conventions do not match those of the evaluation benchmark.

4.4.1 Format-Aligned Fine-Tuning

The format mismatch identified in Config CD+FT is addressed by rewriting the training data preprocessor to produce assistant-turn responses that match the inference pipeline’s argument extraction prompt exactly. Two variants are evaluated: Config FT-aligned-ng, which applies the format-aligned adapter without constrained decoding, and Config CD+FT-aligned, which combines the adapter with the full constrained decoding pipeline.

Config FT-aligned-ng achieves 13.25% accuracy (53/400), a decrease of 0.5 pp relative to Config FT-only. The format-aligned training objective optimises for JSON-only argument extraction and omits the function name from the assistant turn, as required by the guided inference pipeline. The unguided evaluator must recover the function name from the raw output; because the training format no longer produces it, the unguided pass rate decreases. Format alignment improves the guided inference path at the cost of degrading the unguided path.

Fig. 4.4 shows the training loss for both adapters. Both runs use identical hyperparameters and converge to similar final losses (v1: 0.203, v2: 0.229 at step 6750), confirming that the v1 regression is not a training failure. The difference in BFCL accuracy (69.75% vs. 76.75%) is attributable to the training format, not to convergence quality.

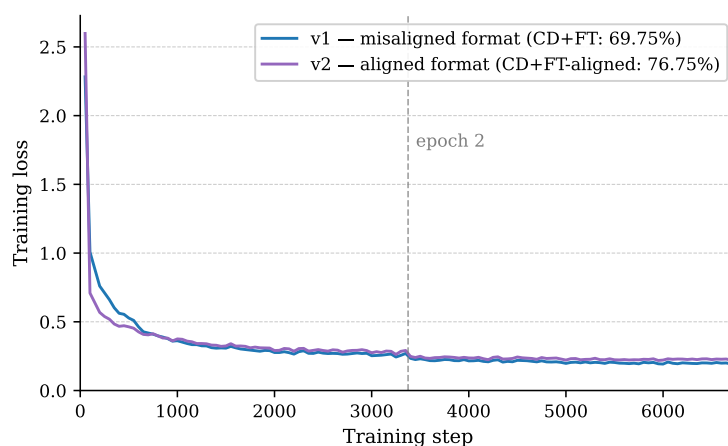


Figure 4.4: Training loss for the misaligned (v1) and format-aligned (v2) LoRA adapters over 6,750 steps (2 epochs on 54,000 xlam examples). Both runs converge to comparable loss values; the 7.0 pp accuracy difference between Config CD+FT and Config CD+FT-aligned is caused by the training output format, not by training quality.

Config CD+FT-aligned achieves 76.75% accuracy (307/400), an improvement of 4.0 pp over

Config CD and 7.0 pp over Config CD+FT. This is the highest accuracy observed across all configurations. Format-aligned fine-tuning breaks the no-training ceiling of 72.75% established by Config CD.

The 23.25% failure rate that remains under Config CD+FT-aligned follows the same taxonomy as Config CD: optional parameter defaults, numeric precision, string format conventions, and enum value mismatches. The xlam training examples do not supply argument values that match BFCL ground truth labels closely enough to eliminate these failure categories. The 4.0 pp gain over Config CD reflects a reduction in error frequency, not the introduction of a new success mechanism.

Table 4.3 presents all ten configurations ordered by accuracy, and Fig. 4.5 shows the same data as a bar chart with frontier reference lines.

Table 4.3: BFCL v4 simple_python AST accuracy across all ten configurations. Δ is relative to Config CD (72.75%).

Config	Technique	Accuracy ($\uparrow$)	Δ vs CD
B	Raw baseline	1.50%	−71.25 pp
FT-aligned-ng	Format-aligned LoRA, no CD	13.25%	−59.50 pp
FT-only	LoRA, no CD	13.75%	−59.00 pp
CD+Q+RAG	CD + AWQ + RAG top-5	47.75%	−25.00 pp
CD+Q+ITC	CD + AWQ + CoT	65.50%	−7.25 pp
CD+FT	CD + LoRA (misaligned)	69.75%	−3.00 pp
PE	CD + few-shot	70.25%	−2.50 pp
CD+Q	CD + AWQ INT4	72.25%	−0.50 pp
CD	Constrained decoding	72.75%	—
CD+FT-aligned	CD + format-aligned LoRA	76.75%	+4.00 pp

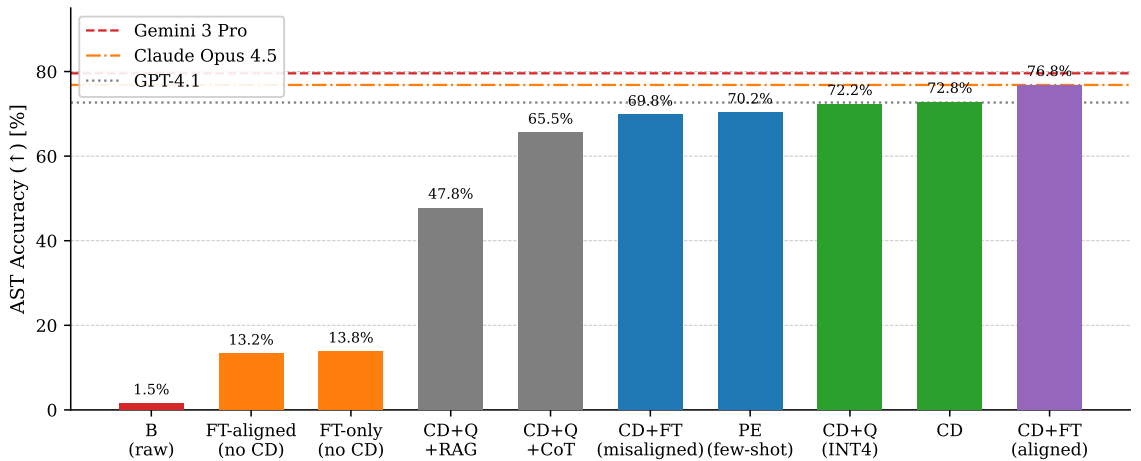


Figure 4.5: BFCL v4 simple_python AST accuracy for all ten configurations. Dashed reference lines show Gemini 3 Pro (79.58%), Claude Opus 4.5 (76.83%), and GPT-4.1 (72.67%). Config CD+FT-aligned (purple) is the only configuration to exceed the no-training ceiling established by Config CD.

4.5 Comparison with Frontier Models

Table 4.4 compares the Qwen 2.5 7B-Instruct results against frontier models, using official scores from the BFCL v4 leaderboard¹ for the Non-Live Simple AST category.

Table 4.4: BFCL v4 Simple Function AST accuracy: Qwen 2.5 7B with constrained decoding vs. frontier models (official leaderboard scores).

Model	Type	Accuracy ($\uparrow$)
Gemini 3 Pro	Frontier	79.58%
Claude Opus 4.5	Frontier	76.83%
GPT-4.1-mini	Frontier	73.33%
GPT-4.1	Frontier	72.67%
Claude Sonnet 4.5	Frontier	72.58%
Claude Haiku 4.5	Frontier	71.00%
GPT-5-mini	Frontier	59.92%
Qwen 2.5 7B + CD+FT-aligned	SLM (ours)	76.75%
Qwen 2.5 7B + CD	SLM (ours)	72.75%
Qwen 2.5 7B + PE	SLM (ours)	70.25%
Qwen 2.5 7B (raw)	SLM (ours)	1.50%

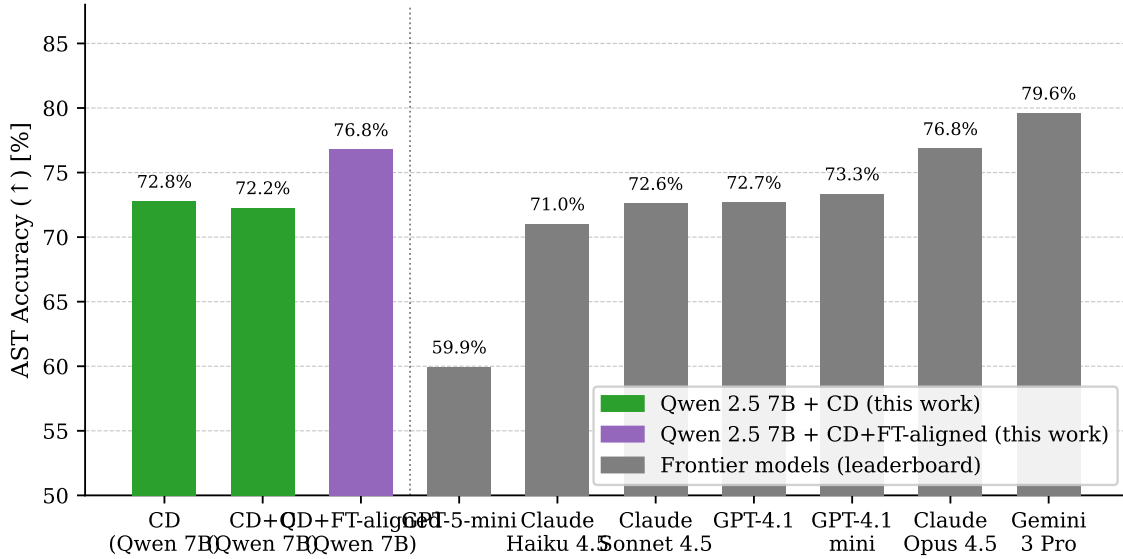


Figure 4.6: BFCL v4 simple_python AST accuracy for Qwen 2.5 7B configurations (this work) vs. frontier models from the official leaderboard (accessed April 2026). Config CD+FT-aligned reaches 76.75%, within 0.08 pp of Claude Opus 4.5 (76.83%).

Fig. 4.6 and Table 4.4 compare the Qwen 2.5 7B results against the leaderboard. Qwen 2.5 7B-Instruct with constrained decoding alone (72.75%) matches or exceeds several frontier models on Simple Function accuracy, including GPT-4.1 (72.67%), Claude Sonnet 4.5 (72.58%), and Claude Haiku 4.5 (71.00%). With format-aligned fine-tuning, Config CD+FT-aligned reaches 76.75%, within 0.08 pp of Claude Opus 4.5 (76.83%). The gap to the best frontier model (Gemini 3 Pro at 79.58%) is 2.83 percentage points.

This suggests that for single-call function calling with a well-defined schema, constrained

¹<https://gorilla.cs.berkeley.edu/leaderboard.html>, accessed April 2026.

decoding on a 7B model closes the gap to frontier-level performance. The remaining gap is driven by argument value accuracy, not format compliance or function selection—the same bottleneck that limits the frontier models themselves.

However, this result requires careful interpretation. The `simple_python` category is the least demanding BFCL scenario: single-turn, single-function calls with no selection ambiguity (one candidate function per test case). Both constrained decoding and frontier models’ native tool-calling APIs guarantee valid JSON and valid function names, levelling the playing field on format compliance. The only differentiator is argument value accuracy, where a 7B model can be competitive on straightforward lookups.

Frontier models distinguish themselves on harder tasks. Their BFCL overall scores, which weight multi-turn (30%) and agentic tasks (40%), remain substantially higher than what an SLM would likely achieve on those categories. Claude Opus 4.5 maintains 77.47% overall, while GPT-4.1 drops to 53.96%, indicating that even among frontier models, simple function calling does not predict general agentic capability. The multi-step evaluation in Section 4.6 quantifies where the SLM–frontier gap re-emerges.

4.6 Agentic Evaluation

Single-call tool accuracy measures one dimension of agent capability. The τ -bench retail benchmark evaluates end-to-end task completion across multi-turn interactions, where the model must select tools, observe results, and iterate until a retail service task is resolved [26]. Config CD is evaluated on the retail domain test split (115 tasks) to establish the agentic baseline.

Three full runs are completed to account for user simulator variance. Pass rates of 4.35%, 5.22%, and 3.48% are observed for runs 1, 2, and 3 respectively, with a mean of 4.35% (5/115). The user simulator (Qwen 2.5 7B on the same vLLM instance) generates stochastic responses despite the model running at temperature 0.0, producing the observed inter-run variance. At $n = 115$ binary tasks the standard error is approximately 1.9 pp; the observed range of 3.5–5.2% is within this noise band.

Scoring is binary: a task passes only if the final database state exactly matches the ground truth at the end of the interaction. Partial completion yields reward 0.0. This criterion reflects the operational requirement that retail service tasks must be resolved completely; partial execution has no customer-facing value.

Table 4.5 places the single-call and multi-step results side by side.

Table 4.5: Single-call accuracy (BFCL) vs. multi-step pass rate (τ -bench) for Config CD on Qwen 2.5 7B-Instruct.

Benchmark	Setting	Pass rate ($\uparrow$)
BFCL <code>simple_python</code>	Single call, 1 turn	72.75%
τ -bench retail	Multi-step, 5–15 turns	4.35%

The 68.4 pp gap between the two benchmarks is the central result of this section. A model that generates individually valid tool calls 72.75% of the time achieves end-to-end task success only 4.35% of the time on the retail domain.

Four failure categories are identified by trajectory inspection. State drift is the most frequent: the model does not track which items have been modified across turns when a request involves multiple sub-tasks. Premature termination is observed when the model

calls `finish` before all sub-tasks are resolved, particularly on multi-item requests. Tool parameter errors persist despite Config CD achieving near-perfect function selection on BFCL, because τ -bench tools accept free-form string identifiers that constrained decoding does not validate. Context management failures are also observed in tasks with long conversation histories, where earlier tool call results are displaced from the model’s working context.

The 4.35% pass rate on τ -bench establishes the agentic baseline for Config CD on the retail domain. The 68.4 pp gap quantifies the boundary separating single-call function calling from reliable multi-step agentic task completion.

4.7 Model-Size Scaling

Table 4.6 reports BFCL v4 `simple_python` accuracy for Config B and Config CD across all four Qwen 2.5 Instruct sizes.

Table 4.6: BFCL v4 `simple_python` AST accuracy for Config B and Config CD across Qwen 2.5 Instruct model sizes (400 test cases).

Model size	Config B	Config CD	CD gain (pp)
0.5B	3.50% (14/400)	51.50% (206/400)	+48.0
1.5B	4.75% (19/400)	62.25% (249/400)	+57.5
3B	2.75% (11/400)	64.75% (259/400)	+62.0
7B	1.50% (6/400)	72.75% (291/400)	+71.25

Config B accuracy is near-zero across all model sizes, ranging from 1.50% to 4.75%, with no consistent trend with scale. The 3B model achieves 2.75%, lower than both the 1.5B (4.75%) and 0.5B (3.50%) models. This confirms that the baseline failure mode is format non-compliance rather than semantic understanding: increasing model size alone does not recover structured output generation without an explicit constraint mechanism.

Config CD accuracy scales consistently with model size. The 0.5B model achieves 51.50%, the 1.5B achieves 62.25%, the 3B achieves 64.75%, and the 7B achieves 72.75%. The CD gain over Config B also increases with model size, from 48.0 pp at 0.5B to 71.25 pp at 7B. This pattern reflects two compounding factors: larger models produce fewer argument value errors once format compliance is enforced, and they benefit more from constrained decoding because a larger fraction of their latent capability is unlocked by removing the format barrier. The 0.5B model already reaches 51.50% under Config CD, approaching GPT-5-mini (59.92%), while the 7B model exceeds GPT-4.1 (72.67%).

4.8 Extended Function-Calling Categories

Table 4.7 reports results for the `multiple_function` and `parallel_function` BFCL categories, evaluated on Qwen 2.5 7B-Instruct-AWQ under Config B and Config CD+Q.

Table 4.7: BFCL v4 AST accuracy on harder categories for Config B and Config CD+Q (Qwen 2.5 7B-Instruct-AWQ, 200 test cases per category).

Category	Config B	Config CD+Q
<code>multiple_function</code>	0.0% (0/200)	0.0% (0/200)
<code>parallel_function</code>	0.0% (0/200)	0.0% (0/200)

Both configurations produce 0.0% accuracy on both categories. Inspection of the raw predictions shows that the pipeline outputs an empty list for every test case. The cause is architectural: the two-stage pipeline is designed for single-function selection in Stage 1 and single-call argument extraction in Stage 2. The `parallel_function` category requires generating multiple function calls simultaneously, which the pipeline cannot produce because Stage 2 is constrained to a single JSON object conforming to one function schema. The `multiple_function` category requires selecting one function from several candidates; while `guided_choice` supports this in principle, the pipeline does not produce output in the format expected by the BFCL evaluator for this category.

This negative result defines the boundary of the current pipeline architecture. The optimization techniques evaluated in this thesis are all applied within the single-call pipeline design. None address the structural limitation of producing multiple simultaneous function calls. Extending the pipeline to support parallel function calling would require a different Stage 2 constraint: a JSON array of objects each conforming to a separate function schema. This extension is left for future work.

4.9 Quantized Fine-Tuning

5 Discussion

5.1 Impact of Constrained Decoding vs. Model Scale

The 71.25 percentage point gap between Config B and Config CD is almost entirely a format compliance gap. Of the 394 failures in Config B, virtually all are structural: the model produces conversational text, multiple JSON objects, or empty output rather than a single valid function call. The six correct outputs in Config B are semantically accurate but happen to be formatted correctly by chance. The model is not unable to reason about the queries; it cannot express its answers in the required structured format without guidance.

This distinction has a direct consequence for interpreting the frontier comparison. Config CD achieves 72.75% on BFCL v4 `simple_python`, matching GPT-4.1 (72.67%) and Claude Sonnet 4.5 (72.58%), and exceeding Claude Haiku 4.5 (71.00%) [25]. The primary mechanism is that constrained decoding enforces valid JSON and valid function names by construction, which is precisely what frontier models’ native tool-calling APIs also do. Once format compliance is equalized, a 7B model and a frontier model are distinguished only by argument value accuracy, where a smaller model can remain competitive on straightforward single-function queries.

The comparison is specific to the `simple_python` category. Each test case provides exactly one candidate function, eliminating function selection as a source of error. With one candidate and `guided_choice`, selection accuracy is 100% regardless of model size. The remaining 27.25% failure rate is entirely in argument values: wrong optional parameter defaults, numeric precision errors, and string format mismatches. Frontier models make the same class of errors; the gap of 6.83 percentage points to Gemini 3 Pro (79.58%) reflects a higher frequency of correct value conventions in larger models, not a qualitatively different failure mode.

The implication for RQ1 is that the raw unoptimized gap of approximately 98.5 percentage points (Config B vs. frontier) collapses to approximately 7 percentage points with constrained decoding alone, on this benchmark category. Scale is not the bottleneck for format compliance. The residual gap is a semantic problem, attributable to the model’s learned associations between function schema contexts and specific value forms.

5.2 Cost-Benefit Analysis of Each Method

Constrained decoding yields the highest return of any technique evaluated. A 71.25 percentage point accuracy gain is achieved with no additional inference cost beyond the guided decoding schema: no extra parameters, no additional tokens, and no training. The gain is entirely structural: logit masking at each decoding step eliminates the malformed output that accounts for 98.5% of failures in Config B [23, 11].

AWQ INT4 quantization adds a further infrastructure benefit at near-zero accuracy cost [14]. The -0.5 percentage point change (two test cases) falls within run-to-run variance. Memory is reduced by 63.5% (14.25 GiB to 5.20 GiB) and model loading time by 83.5% (212 s to 35 s). The cost-benefit ratio is strongly positive: substantial deployment benefits at negligible accuracy cost.

Few-shot prompting (Config PE) and chain-of-thought reasoning (Config CD+Q+ITC) both produce negative outcomes [3, 22]. Config PE reduces accuracy by 2.5 percentage points with a context overhead of approximately 300 tokens per query. Config CD+Q+ITC

reduces accuracy by 6.75 percentage points while approximately doubling per-case latency. Both techniques add cost and reduce accuracy. The failure mechanism is the same in both cases: the residual 27% failure rate after constrained decoding is caused by a mismatch between the model’s learned value priors and the benchmark’s ground truth conventions. Prompting interventions that ask the model to reason more explicitly or reference examples move it further from the benchmark’s arbitrary conventions rather than toward them. Two independent negative results with the same directional outcome and a shared mechanistic explanation establish that prompting cannot address this failure class.

This finding aligns with the scope conditions reported by Wei et al. [22]: chain-of-thought prompting improves performance on tasks that require multi-step arithmetic decomposition or symbolic manipulation, but not on tasks where the bottleneck is value recall rather than reasoning. Argument extraction from a function schema is not a reasoning problem in that sense: the model either has the correct value convention for a given schema context in its parametric knowledge or it does not. An explicit reasoning step directs additional compute at the wrong subproblem. Snell et al. [20] generalise this point in the context of inference-time scaling: additional compute at generation time improves performance when directed at tasks that benefit from systematic search or decomposition. Single-call argument extraction with a constrained output space does not have that structure, which explains why chain-of-thought reasoning reduces rather than improves accuracy in this setting.

Retrieval-augmented generation produces the largest accuracy drop: -24 percentage points relative to Config CD+Q, despite a retrieval recall@5 of 97.2% [13]. The bottleneck is not retrieval quality but candidate disambiguation. When multiple semantically similar functions are retrieved, the 7B model cannot reliably identify the one matching the user’s intent. The infrastructure cost of RAG is substantial (an offline FAISS index and an embedder at query time), making its negative accuracy outcome particularly unfavorable.

The disambiguation failure identified here is avoided by an alternative design in Gorilla [16], which incorporates retrieval at training time rather than inference time. By generating training examples from retrieved API documentation, Gorilla teaches the model to use retrieved content as a grounding signal without exposing it to multiple similar candidates at query time. This sidesteps the inference-time disambiguation problem entirely, at the cost of coupling the model to a fixed API catalog. The configuration evaluated in this thesis uses semantic similarity retrieval at inference time with no training signal on candidate disambiguation, which may explain the low selection accuracy despite high retrieval recall@5 (97.2%). The 66% of RAG failures that produce answers identical to non-RAG predictions (from the RAG failure breakdown in Fig. 4.3) confirms that retrieval adds overhead without changing the model’s output for the majority of cases. A design that combined inference-time retrieval with a fine-tuning signal on tool selection from retrieved candidates would be a natural next step.

LoRA fine-tuning on the `xlam-function-calling-60k` dataset [28, 8] produces a regression of 3 percentage points under Config CD+FT relative to Config CD. The regression is attributed to a format mismatch between the training distribution, which encodes assistant turns as Python call syntax, and the evaluation pipeline, which expects a JSON argument object. The technique is not ruled out by this result. The format mismatch hypothesis is confirmed by Config CD+FT-aligned, which retrains on the same xlam content reformatted to match the inference pipeline’s JSON output convention exactly. Config CD+FT-aligned achieves 76.75% (307/400), a gain of 7.00 percentage points over Config CD+FT v1 and 4.00 percentage points above the Config CD baseline, making it the best result in the

evaluation. Config FT-only confirms that fine-tuning does improve format compliance in the unguided setting (+12.25 percentage points over Config B), establishing that the training signal is meaningful. The 63 percentage point gap between Config FT-aligned-ng (13.25%) and Config CD+FT-aligned confirms that constrained decoding remains the dominant contributor even after fine-tuning.

Taken together, the results support a clear ordering by cost-benefit ratio for this task: constrained decoding first, quantization second, and training-based methods as the remaining path to accuracy above the no-training ceiling. Prompting interventions offer no benefit on this failure class and add latency or context overhead.

5.3 Limitations

The results are based on the `simple_python` category of BFCL v4, which is the least demanding of the four BFCL categories [25]. Each test case consists of a single user turn, a single candidate function, and a single expected call. The constrained decoding advantage depends on the oracle function provision: `guided_choice` guarantees correct function selection only when the correct function is the sole candidate. The RAG results (47.75% at top-5) show that this assumption fails under realistic conditions. Generalisation to harder BFCL categories (multiple function candidates, parallel function calls, multi-turn interactions) is not established and may not hold.

The full configuration ladder is evaluated only on the 7B model. Smaller Qwen 2.5 sizes (0.5B, 1.5B, 3B) were not evaluated across all configurations, so the interaction between model scale and each optimization technique is not characterised. Whether constrained decoding closes the frontier gap equally at smaller scales, or whether there is a scale threshold below which semantic failures dominate, remains an open question.

The regression in Config CD+FT v1 is attributed to the format mismatch between the xlam training distribution and the evaluation pipeline. This attribution is confirmed by the controlled experiment: Config CD+FT-aligned holds all variables fixed and changes only the training output format, recovering to 76.75% and surpassing Config CD by 4.00 percentage points. The v1 regression is therefore attributable to the training format, not to the xlam content or to a fundamental incompatibility between fine-tuning and constrained decoding.

The τ -bench retail evaluation reveals a sharp discontinuity between single-call and multi-step performance. Config CD achieves 72.75% on BFCL `simple_python` but only 4.35% pass rate on τ -bench retail, which requires reasoning across 5–15 tool calls with real observations per task. Simple function calling accuracy does not predict multi-step agentic performance; the 7 percentage point frontier gap on the single-call category widens to roughly 95 percentage points on multi-step tasks, where the failure mode shifts from argument value errors to multi-turn planning and error recovery.

The BFCL ground truth encodes specific value conventions that are often arbitrary from a deployment perspective. A model that produces "gasoline" where the ground truth expects "gas" is scored as incorrect despite both being valid in a real API call. The 27% residual failure rate after constrained decoding may overestimate the practical error rate for production use, where value conventions are defined by the API schema rather than by a benchmarking dataset.

5.4 Practical Implications

Two configurations are production-viable on consumer hardware, each addressing a different deployment constraint. Config CD+Q achieves 72.25% AST accuracy on 5.20 GiB of GPU

memory with a 35 s model load time on an NVIDIA L40S. The same configuration fits on a consumer RTX 4090 (24 GiB) with ample remaining capacity for the key-value cache. It requires no training and is deployable immediately from a pre-quantized checkpoint. Config CD+FT-aligned achieves 76.75% on the same hardware footprint (AWQ INT4 weights plus a merged LoRA adapter), reducing the residual failure rate from 27.75% to 23.25% at the cost of a fine-tuning run on aligned training data.

In a cascade architecture where an SLM handles routine tool-calling requests and escalates to a frontier model when uncertain, Config CD+Q defines the no-training operating point: approximately 72% of requests are handled correctly at 5.20 GiB memory, with the remaining 28% escalated. Config CD+FT-aligned extends this operating region to approximately 77%, reducing escalation traffic by roughly 4 percentage points relative to Config CD+Q. The escalation boundary in both cases corresponds to the same failure classes: wrong optional parameter defaults, numeric precision mismatches, and string format conventions. Queries with explicit argument values and unambiguous types are reliably handled at the SLM tier; queries involving domain-specific numeric precision or underspecified optional parameters remain candidates for escalation.

The routing decision in the cascade tier determines which queries reach the frontier model. Structural validation – checking that the output is valid JSON and names a known function – is satisfied by every output under Config CD and above, and therefore cannot serve as a routing gate after constrained decoding is applied. Two alternative routing signals are more appropriate. Query-level classification, applied before inference, estimates whether a query is likely to involve underspecified optional parameters or domain-specific numeric conventions, and routes predicted-hard queries directly to the frontier tier before the SLM is invoked. Confidence-based routing, applied after inference, uses the token-level probability of the generated argument values as a proxy for answer reliability, and escalates low-confidence outputs. Neither signal is evaluated in this thesis; both are viable directions for future work that the current residual failure analysis motivates.

The economic argument for the cascade tier can be quantified directly from these results. If frontier model calls cost c per query and the SLM tier cost is approximated as zero per query (amortized over hardware), a cascade in which the SLM correctly handles a fraction p of requests saves $p \cdot c$ per query relative to routing everything to the frontier model. Config CD+Q achieves $p \approx 0.72$, and Config CD+FT-aligned achieves $p \approx 0.77$. The 4.0 percentage point gain from fine-tuning translates directly to a 4.0 percentage point reduction in frontier API traffic. Whether the fine-tuning cost is recovered depends on query volume: at high throughput, the API cost reduction per query accumulates rapidly; at low throughput, the no-training configuration is preferable.

Chain-of-thought reasoning is contraindicated at the SLM tier. Config CD+Q+ITC raises the failure rate by 6.75 percentage points relative to Config CD+Q, increasing escalation traffic rather than reducing it [22]. The mechanism is structural: the reasoning step moves the model toward linguistically natural value forms that differ from the conventions the ground truth encodes.

For tool catalogs with many functions, RAG-based tool selection requires a disambiguation capability that the 7B model does not have at top-5 retrieval [13]. A more viable configuration would use top-1 or top-2 retrieval, a re-ranking step before the selection stage, or a fine-tuning signal on tool-selection examples to reduce the frequency of sibling-function confusion.

Fine-tuning remains the most direct path to accuracy above the 72–73% no-training ceiling.

The key constraint is training data alignment: the training format must match the inference pipeline’s prompt structure and output conventions exactly. The v1 xlam experiment shows that a general-purpose function-calling dataset with a different output convention actively degrades performance under constrained decoding. A production fine-tuning pipeline would require training examples generated from the target deployment’s own tool schemas and argument conventions, not from a general-purpose corpus.

6 Conclusion

6.1 Answering the Research Questions

RQ1. The unoptimized performance gap between Qwen 2.5 7B-Instruct and frontier models on BFCL v4 `simple_python` is approximately 98.5 percentage points: the base model (Config B) achieves 1.5%, while frontier models range from 71.00% (Claude Haiku 4.5) to 79.58% (Gemini 3 Pro) [25]. This gap is almost entirely a format compliance gap. The base model understands the queries but cannot produce structured output without guidance: 394 of 400 failures in Config B are malformed outputs rather than semantically wrong answers. Constrained decoding alone (Config CD) closes the gap to approximately 7 percentage points (72.75% vs. 79.58%), matching GPT-4.1 (72.67%) and Claude Sonnet 4.5 (72.58%). The residual gap is in argument value accuracy, where frontier models have a moderate advantage attributable to stronger learned associations for specific numeric, string, and format conventions.

RQ2. Constrained decoding is the dominant optimization technique, contributing +71.25 percentage points at zero additional inference cost. AWQ INT4 quantization reduces model memory by 63.5% (14.25 GiB to 5.20 GiB) and startup time by 83.5% (212 s to 35 s) at a cost of -0.5 percentage points, within run-to-run variance [14]. Both prompting interventions evaluated produced negative outcomes: few-shot prompting (Config PE) reduced accuracy by 2.5 percentage points [3], and chain-of-thought reasoning (Config CD+Q+ITC) reduced accuracy by 6.75 percentage points at approximately double the per-case latency [22]. Retrieval-augmented generation (Config CD+Q+RAG) reduced accuracy by 24 percentage points despite a retrieval recall@5 of 97.2%, with the loss attributable to function disambiguation failure rather than retrieval quality [13]. LoRA fine-tuning on `xlam-function-calling-60k` [28, 8] produced a 3 percentage point regression under Config CD+FT relative to Config CD, attributed to a format mismatch between the training distribution and the evaluation pipeline. The format mismatch hypothesis is confirmed by Config CD+FT-aligned, which reformats training outputs to match the inference pipeline exactly and achieves 76.75% (307/400), a gain of 4.00 percentage points above Config CD and the best result in the evaluation. The finding for RQ2 is that the residual 27% failure rate after constrained decoding cannot be addressed by prompting: these errors are in the model’s learned value priors, and only training-based interventions with aligned format can change them.

RQ3. Two configurations achieve production-viable accuracy on consumer hardware. Config CD+Q achieves 72.25% on 5.20 GiB of GPU memory with no training required, fitting on a consumer NVIDIA RTX 4090 (24 GiB) with substantial remaining capacity for the key-value cache. Config CD+FT-aligned achieves 76.75% on the same hardware footprint, with the LoRA adapter merged into the base weights before quantization. For the `simple_python` use case, a fully optimized SLM configuration is production-viable on consumer hardware, with training data alignment as the binding constraint on the achievable accuracy ceiling.

6.2 Future Work

The τ -bench result (4.35% pass rate on retail) establishes that single-call accuracy does not predict multi-step agentic performance. Improving multi-step reliability is the most

important open direction. The failure mode on τ -bench is not argument formatting – constrained decoding handles that – but multi-turn planning and error recovery across 5–15 tool calls. Techniques such as process reward models, chain-of-thought with tool-observation grounding, and task-specific fine-tuning on multi-turn trajectories are natural next steps [26].

The full configuration ladder has been evaluated in depth only on the 7B model. Evaluating the same configurations on Qwen 2.5 sizes 0.5B, 1.5B, and 3B would characterise the interaction between model scale and each optimization technique, and identify whether there is a minimum scale threshold below which constrained decoding no longer closes the frontier gap.

RAG-based tool selection requires a stronger disambiguation signal than top-5 retrieval provides for a 7B model. Three concrete directions are: reducing k to 1 or 2 to narrow the candidate set at the cost of recall; adding a re-ranking step between retrieval and selection to order candidates by query-specific relevance; and fine-tuning the model on tool-selection examples from the target catalog to improve sibling-function discrimination.

The constrained decoding advantage is established on `simple_python`, where one candidate function is provided per test case. Evaluating Config CD and Config CD+Q on the `multiple_function` and `parallel_function` BFCL categories would test whether the advantage holds when the model must select from multiple candidates or produce parallel calls, conditions closer to a production tool catalog.

Training data alignment is the binding constraint on LoRA fine-tuning for this task. Generating training examples from the target deployment’s own tool schemas and argument conventions, rather than from a general-purpose corpus such as xlam, would remove the format mismatch as a confound and provide a direct learning signal for the value conventions that the no-training configurations fail on. Such data could be generated synthetically from the BFCL training split or from API documentation of the production tool catalog.

Bibliography

- [1] Marah Abdin et al. “Phi-4 Technical Report”. In: *arXiv preprint arXiv:2412.08905* (2024).
- [2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. “Prompting Is Programming: A Query Language for Large Language Models”. In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023).
- [3] Tom Brown et al. “Language Models are Few-Shot Learners”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [4] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized Language Models”. In: *Advances in Neural Information Processing Systems* 36 (2023).
- [5] Aaron Grattafiori et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024).
- [6] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. “Distilling the Knowledge in a Neural Network”. In: *arXiv preprint arXiv:1503.02531* (2015).
- [7] Jordan Hoffmann et al. “Training Compute-Optimal Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.
- [8] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *arXiv preprint arXiv:2106.09685* (2022).
- [9] International Committee of Medical Journal Editors (ICMJE) et al. *Recommendations for the Conduct, Reporting, Editing, and Publication of Scholarly Work in Medical Journals*. <https://www.icmje.org/icmje-recommendations.pdf>. Originally prepared by Severinsen and Ekern (2017), updated by Torp (2022). Translated by Carole Hognestad. Aug. 2020. URL: <https://www.icmje.org/icmje-recommendations.pdf>.
- [10] Jared Kaplan et al. “Scaling Laws for Neural Language Models”. In: *arXiv preprint arXiv:2001.08361* (2020).
- [11] Woosuk Kwon et al. “Efficient Memory Management for Large Language Model Serving with PagedAttention”. In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*. 2023.
- [12] Philipp Lehman et al. *Biblatex – Sophisticated Bibliographies in LaTeX*. 2018. URL: <https://www.ctan.org/pkg/biblatex>.
- [13] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020).
- [14] Ji Lin et al. “AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration”. In: *Proceedings of Machine Learning and Systems*. 2024.
- [15] OpenAI. “GPT-4 Technical Report”. In: *arXiv preprint arXiv:2303.08774* (2023).
- [16] Shishir G Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv preprint arXiv:2305.15334* (2023).
- [17] Yujia Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *arXiv preprint arXiv:2307.16789* (2024).
- [18] Qwen Team. “Qwen2.5 Technical Report”. In: *arXiv preprint arXiv:2412.15115* (2024).
- [19] Raghav Sharma et al. *Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade-offs*. 2025. arXiv: 2505.00749 [cs.AI]. URL: <https://arxiv.org/abs/2505.00749>.
- [20] Charlie Snell et al. “Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters”. In: *arXiv preprint arXiv:2408.03314* (2024).

- [21] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
- [22] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems* 35 (2022).
- [23] Brandon T Willard and Rémi Louf. “Efficient Guided Generation for Large Language Models”. In: *arXiv preprint arXiv:2307.09702* (2023).
- [24] Brandon T Willard and Rémi Louf. *Outlines: Structured Text Generation*. 2024. URL: <https://github.com/dottxt-ai/outlines>.
- [25] Fanjia Yan et al. *Berkeley Function Calling Leaderboard*. 2024. URL: <https://gorilla.cs.berkeley.edu/leaderboard.html>.
- [26] Shunyu Yao et al. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. 2024. arXiv: 2406.12045 [cs.AI]. URL: <https://arxiv.org/abs/2406.12045>.
- [27] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629* (2023).
- [28] Jianguo Zhang et al. “xLAM: A Family of Large Action Models to Empower AI Agent Systems”. In: *arXiv preprint arXiv:2409.03215* (2024).

A Generative AI Disclosure

In accordance with DTU’s guidelines on the use of generative AI tools in academic work, the following AI-assisted tools were used during the preparation of this thesis.

Claude Code (Anthropic) was used as a coding assistant during the implementation of the evaluation pipeline, HPC job scripts, and data processing scripts. Generated code was reviewed, tested, and modified before use. No generated code was submitted without verification against expected outputs.

Claude (Anthropic, claude.ai) was used as a writing assistant to produce draft text for thesis sections. All drafted text was reviewed, revised, and rewritten in the author’s own voice before inclusion. The author is responsible for all factual claims, experimental results, and interpretations presented in this thesis.

No AI tool was used to generate experimental data, benchmark results, or citations. All numerical results reported in this thesis were produced by running experiments on the DTU HPC cluster and collected by the author.

AI tools are not listed as authors or co-authors of this work, in accordance with the Vancouver Convention as adopted by DTU [9].

B Hyperparameters and Training Details

Table B.1 lists the hyperparameters used for LoRA fine-tuning in all training configurations.

Table B.1: LoRA fine-tuning hyperparameters for all training configurations.

Parameter	Value
Base model	Qwen/Qwen2.5-7B-Instruct
Dataset	Salesforce/xlam-function-calling-60k
Training examples	54,000
Validation examples	6,000
LoRA rank	16
LoRA alpha	32
Target modules	q_proj, v_proj
Dropout	0.05
Epochs	2
Batch size	4
Gradient accumulation steps	4
Learning rate	2×10^{-4}
LR scheduler	cosine
Dtype	bfloat16
Optimizer	AdamW
Max sequence length	2048

C Full Benchmark Results

Table C.1 reports the complete per-configuration results on BFCL v4 `simple.python` (400 test cases).

Table C.1: Full BFCL v4 Simple Function AST accuracy results across all configurations evaluated in this thesis.

Config	Description	Correct	Accuracy ($\uparrow$)	Delta vs CD
B	No constrained decoding	6/400	1.50%	−71.25 pp
PE	CD + few-shot (3 examples)	281/400	70.25%	−2.50 pp
CD	Constrained decoding	291/400	72.75%	baseline
CD+Q	CD + AWQ INT4	289/400	72.25%	−0.50 pp
CD+Q+ITC	CD+Q + chain-of-thought	262/400	65.50%	−7.25 pp
CD+Q+RAG	CD+Q + FAISS top-5	191/400	47.75%	−25.00 pp
FT-only	LoRA merged, no CD	55/400	13.75%	−59.00 pp
CD+FT	CD + LoRA merged (v1)	279/400	69.75%	−3.00 pp
FT-aligned-ng	Format-aligned LoRA, no CD	53/400	13.25%	−59.50 pp
CD+FT-aligned	CD + format-aligned LoRA	307/400	76.75%	+4.00 pp

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk